

ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
"НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
"ВЫСШАЯ ШКОЛА ЭКОНОМИКИ"

НЕГОСУДАРСТВЕННОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
"РОССИЙСКАЯ ЭКОНОМИЧЕСКАЯ ШКОЛА" (ИНСТИТУТ)

Выпускная Квалификационная Работа

Могут ли метрики микроструктуры рынка улучшить обнаружение мошенничества с выводом ликвидности?

Бакалаврская программа

Совместная программа по экономике НИУ ВШЭ и РЭШ

Автор:

Н.И. Ключников

Руководитель ВКР:

А.В. Колоколов

Москва, 2026 г.

Can Market Microstructure Metrics Enhance Rug Pull Detection?

Аннотация

Несмотря на то, что мошенничество с выводом ликвидности является одной из крупнейших угроз на криптовалютных рынках, было проведено лишь несколько исследований с целью построения моделей раннего обнаружения мошеннических токенов. Я провёл подробное исследование в обнаружении данного типа мошенничества на децентрализованной бирже Raydium блокчейна Solana, проверяя, способны ли меры микроструктуры рынка улучшить определение мошеннических токенов. Проанализировав 32426 токенов, я провёл A/B-тестирование метрик эффективности модели до и после добавления 7 признаков, таких как лямбда Кайла, мера неликвидности Амихуда и VPIN (вероятность информированной торговли). Анализ показал, что эти признаки способны немного, но статистически значимо улучшить эффективность предсказания, частично в соответствии с теорией микроструктуры рынка. Данные результаты представлены с учётом выбора модели, подбора гиперпараметров и подхода к разметке токенов. Кроме того, данное исследование предлагает новую адаптацию мер рыночной микроструктуры для децентрализованных бирж и представляет практическую пользу для оценки рисков в реальном времени в системах децентрализованной торговли.

Abstract

While rug pulls are among the biggest threats in cryptocurrency markets, only a few studies have attempted to build early detection models for malicious tokens. I conducted a comprehensive rug pull detection study on Solana's Raydium DEX, testing whether market microstructure measures can enhance identification of malicious coins. With 32,426 token launches analyzed, I conducted an A/B test on model performance before and after adding 7 features, including Kyle's lambda, Amihud's illiquidity, VPIN and others. The analysis suggests that these features can lead to an incremental, yet statistically significant improvement in performance, partly consistent with microstructure theory. Findings are reported with robustness to choice of model, hyperparameter tuning and labeling approach. Additionally, this study provides a novel adaptation of microstructure metrics for DEX data and offers practical utility for real-time risk assessment in DeFi trading systems.

Contents

1	Introduction	3
2	Literature Review	4
2.1	Rug Pull Detection on Decentralized Exchanges	4
2.2	Machine Learning in Financial Fraud Detection	7
2.3	Market Microstructure Metrics as Predictors	8
3	Data	11
3.1	Scope of Research	11
3.2	Data Collection	12
4	Methodology	13
4.1	Problem Specification and Targets	13
4.2	Data Engineering	16
4.3	Features	17
4.3.1	Benchmark	17
4.3.2	Market Microstructure Measures	18
4.4	Identification of Model Improvement	21
5	Results	23
5.1	Change in Model Performance	23
5.2	Interpretation	26
6	Conclusion	31
	References	33
	Appendices	39
A	Benchmark Features	39
B	SHAP with has_pumpdotfun_history	40
C	Hyperparameters	41
D	Robustness	41
D.1	Statistical tests of performance metric change	41
D.2	Cross-sectional Feature Construction	43
D.3	Model Calibration and Correctness of F1 score	46

1 Introduction

With the development of cryptocurrency markets and rising popularity of decentralized exchanges (DEXs), many traders have fallen victim to new forms of financial fraud. Among those, one of the more prevalent types is a rug pull scam, where developers abandon a project, escaping with investors' money ([Mazorra et al., 2022](#)). A significant part of the tokens launched in the past few years were made with the sole intent to attract a large number of traders and suddenly remove liquidity, rendering the token worthless ([Kalacheva et al., 2025](#)).

Specifically, one of the major blockchains, Solana, has been affected by this form of fraud. Being one of the most affordable systems in terms of trading and launching tokens, it became the ground for many rug pulls. There have been several high-profile promotions of token launches, with creators promising long-term commitment and benefits for investors, only to quit early and cash out ([Merkle Science, 2025](#)). While such fraud used to be prevalent on the Ethereum and BTC blockchains, more and more influencers have been, intentionally or not, involved in rug pulls on Solana. At the same time Pump.fun, a platform for creating new tokens on Solana, has made it easier than ever for anyone to start their own cryptocurrency project. This allowed many individuals without any prior following to do fraud of their own ([Khalili, 2024, 2025](#)). As a result, Pump.fun has become closely associated with rug pulls ([Solidus Labs Research, 2025](#)).

The negative consequences of rug pull scams are quite evident. They generate significant losses for retail traders, undermine trust in decentralized exchanges and the broader DeFi ecosystem, and can reduce liquidity and participation in the affected markets for extended periods ([Merkle Science, 2025](#); [Solidus Labs Research, 2025](#); [Sun et al., 2024](#)).

Due to the risks that rug pulls pose to DEX traders, a body of research has been conducted in order to analyze this form of fraud, as well as develop ex post and ex ante detection strategies. Some key patterns have been identified. The researchers have shown that transaction event clustering, transfer graph topology, and smart contract contents may foretell an upcoming rug pull. Moreover, the fraud occurs quickly, often within an hour, while effective detection can be done early, in just minutes since the token's launch. Additionally, rug pull patterns may not transfer across DEXs due to various idiosyncratic factors.

Nonetheless, several issues can be identified in the past studies. Firstly, the feature sets used in the past papers were overly simplistic, mostly consisting of simple trade aggregates. Attempts to improve it have not been quite successful, as shown by feature importance. A need for more complex and informative features has been identified in the literature. Secondly, there has been no unified definition of rug pulls established, with many different labeling approaches proposed. There is a need for a scalable solution that improves specificity of rug pull identification, maintaining low false negative rate.

Additionally, despite the prevalence of rug pulls on Solana, there is yet to be a study in this area conducted on it. This blockchain has a significantly different architecture from Ethereum—the platform that was analyzed in most of previous papers. While a significant part of interactions on the Ethereum blockchain are dictated by larger block creation windows and high gas fees (Capponi et al., 2025; Jia, 2024), Solana does not have a public mempool in the first place, while also having high frequency block creation and low transaction costs (Solana Foundation, 2026a, 2019, 2026b). Therefore, it is likely that several findings from prior work do not apply, and a separate study is needed.

This thesis addresses these gaps by 1) conducting the first large-scale rug pull-proxy prediction study focused on the Solana’s Raydium DEX, 2) treating rug pull labels as a family of proxy problems rather than a single ground-truth definition and 3) extending standard feature sets with market microstructure measures adapted to DEX data. To the best of my knowledge, this thesis introduces the first methodological framework for analyzing Solana’s DEX markets by reconstructing TAQ-like time series directly from vault account transfers. This study also demonstrates that some of the well-established market microstructure measures are significantly informative for predicting rug pull-like events, with several patterns documented in classic financial theory transferring to the new types of markets. Proposed adaptations to DEX data, such as extending bid-ask spread measures to negative values, resulted in extending the utility of these metrics beyond the standard applications. The results were tested under several settings in order to ensure the robustness. Overall, this study provides a reproducible pipeline for reconstructing high-frequency AMM trading series from on-chain transfers and building statistically validated early-warning classifiers for rug pull-like adverse events.

The rest of the thesis is structured this way: section 2 reviews related literature, section 3 describes the data used, its scope and the collection process, section 4 introduces proxy targets, feature engineering and the evaluation design, section 5 reports results and interpretation, section 6 concludes the study. The appendix provides additional technical notes and robustness checks.

2 Literature Review

2.1 Rug Pull Detection on Decentralized Exchanges

The first paper to conduct a systematic analysis of rug pulls was Xia et al. (2021). The researchers were able to construct a sizable dataset of malicious and reliable tokens using two key heuristics. First, as the authors claimed, most scam tokens tend to imitate other successful cryptocurrencies and entities. Second, accounts that were involved in a launch of a rug pull coin should be labeled as malicious and all their projects considered rug pulls. To expand their dataset, the researchers used a random forest classifier, achieving a cross-

validated F1-score of 97%. The range of implemented features included time of activity and inactivity, the concentration of transaction events, wallet data, etc. This pioneering study identified several key properties of rug pull scams, demonstrating the importance of studying the transaction event clustering and smart contract code. However, its findings may not be relevant to present-day DEXs, as the research was conducted on data from the early stages of DEX development. Moreover, the heuristics-based identification strategy is inherently likely to overlook a significant portion of scam and non-scam tokens. This means that various properties of rug pulls may have been missed in the analysis. Additionally, researchers claimed that their ML fraud detection algorithm could be used to predict future rug pulls. Yet the features used, although applicable to their ex post analysis, are not available in advance, rendering forecasting infeasible.

In [Mazorra et al. \(2022\)](#) the authors aimed to continue the research commenced in [Xia et al. \(2021\)](#). They set out to develop an ex ante rug pull detection algorithm, as well as characterize a wider range of crypto scams. The research moved away from simple heuristics, as the researchers attempted to define specific types of rug pulls. As such, they identified Simple rug pulls, where the developers burn their liquidity, Sell-type rug pulls, where developers sell off their holdings instead, and Trap Doors, which have intentional vulnerabilities in their smart contracts. The main labeling criterion was the occurrence of an extreme maximum drawdown with no posterior recovery. This classification approach has since served as a key reference for subsequent research ([Kalacheva et al., 2025](#); [Srifa et al., 2025](#); [Wu et al., 2024](#)). To develop the first ML rug pull detector, the authors enhanced transaction features with new ones: the Herfindahl–Hirschman Index of token ownership and the Clustering Coefficient of the transaction graph. As a result, their classifier achieved a maximum F1-score of approximately 82% under their more realistic experimental settings. Overall, this work represents a significant advancement in this area of research, as it addressed the key limitations of [Xia et al. \(2021\)](#) by providing a robust rug pull classification and identification methodology, described the first rug pull forecasting model and proposed new features for ownership concentration and graph topology. Yet, new issues arose with the new data collection approach. As the researchers gathered reliable tokens from major cryptocurrency aggregators and auditors, they effectively labeled solely the large survivors as non-malicious tokens. Meanwhile, by their definition of a rug pull, a marked token may have not been malicious but simply an extreme failure. Therefore, the forecasting problem trivialized to separating top quantile tokens from the bottom quantile. This is supported by their SHAP analysis—the most important feature happened to be the number of transactions, with its larger values corresponding to the tokens that were labeled reliable. Consequently, the middle quantile was excluded from the analysis, introducing both selection and survivorship biases. Moreover, the class imbalance issue was very significant in their dataset (around 600 reliable tokens against 27,000 malicious ones), making the resulting model much less effective.

Finally, the novel features had relatively low importance, while requiring complex and computationally expensive calculations.

A series of new papers were published in the past year in this area of research, trying to expand upon the preceding studies. In [Srifa et al. \(2025\)](#) the authors set out to find the optimal timeframe for rug pull detection. To do so, they modified the labeling strategy from Mazorra et al. (2022) by marking the tokens that remained active as non-malicious, and the inactive coins that had a maximum drawdown of -100% as rug pulls. The authors showed that the best period to predict whether a coin is a scam is in the first hours from its launch. This works as rug pulls occur fast, while even a small period of data collection is sufficient for separating them from non-malicious tokens. However, while some of the issues in the identification strategy from Mazorra et al. (2022) were addressed, this study continues to conflate rug pull detection with distinguishing long-lasting tokens from quickly crashing ones, which misses a spectrum of coin types in between. Additionally, their algorithm relied mainly on simple transaction aggregates as features. Specifically, the outcomes of their study may have been different with more complex features chosen, which is not addressed directly.

The subsequent paper, [Kalacheva et al. \(2025\)](#), applied the findings on rug pull identification timeframes, developing an early rug pull detection algorithm. The authors rejected maximum drawdown as a reliable labeling criterion, substituting it with early lasting inactivity, which accounts for trap door scams. This paper was also the first in this area of research to account for sniping—high-speed automatic arbitrage on newly launched tokens—by using large validator bribes as a proxy for priority-seeking in trade execution. With an addition of code reuse features, their model successfully identified scam tokens within the first 5 minutes of their existence. Nonetheless, the timeframe for data collection is both short and relatively arbitrary, rendering these results unreliable. Despite trying to improve the labeling approach of Mazorra et al. (2022), the researchers introduced even more false positives, marking all insignificant tokens as rug pulls.

In the subsequent pre-print paper, [Yaremus et al. \(2025\)](#), the authors challenged the external validity of previously identified empirical patterns in scam tokens across DEXs and blockchains. They developed a prediction model for the TON blockchain’s Ston.Fi and DeDust DEXs. An important contribution was the direct comparison of the two paradigms in rug pull labeling: Idle-based, focused on a sudden cessation of all trading activity, and TVL-based, which relies on extreme maximum drawdowns and the outflow of liquidity. The authors employed both approaches to label tokens and trained and evaluated a model on each of them. However, their assessment is flawed, as they use unreliable labels to evaluate the models, effectively assuming each of them to be the ground truth, which yields a proxy bias and circular validation. Nonetheless, the researchers demonstrated that simple combination of cross-DEX data for model training leads to low performance, and while crash patterns (TVL-based rug pulls) look similar across

the blockchain, each DEX has unique idle activity. Their analysis indicates that the simple transaction-based feature set makes it difficult to train a generalizable model, with significant signs of overfitting to platforms’ idiosyncrasies.

Several papers have criticized this body of research, predominantly focusing on the foundational Xia et al. (2021) and Mazorra et al. (2022). Specifically, some studies demonstrated that the high metrics that researchers claim in their articles are often significantly overstated, and only their more conservative scores are consistent with actual performance (Wu et al., 2024, 2025). Additionally, they argued that using transaction aggregates as features cannot capture the dynamics of rug pull scams (Wu et al., 2024). For that reason, various alternative approaches have been implemented, including smart contract code analysis (Ma et al., 2023; Lin et al., 2024) and graph neural networks (Wu et al., 2024, 2025). Moreover, researchers point out that identification of rug pulls based on their maximum price/liquidity drop does not account for scams where a mechanism implemented in the smart contract can disallow any trading, effectively defrauding investors without allowing the coin price to move (Kalacheva et al., 2025; Wu et al., 2024).

Overall, there has been significant progress in this area of research. While the general properties of rug pulls have been established and several forecasting and identification methodologies have been proposed, several key issues are still present and the proposed approaches have some common issues. First of all, across the articles researchers have attempted to give a general ground-truth labeling approach for rug pull scams and non-malicious tokens, yet most of the methodologies are prone to false positives. There have been attempts to solve this issue (Kalacheva et al., 2025; Wu et al., 2024; Yaremus et al., 2025), however, the proposed solutions have significant flaws. A new approach is required to eliminate the selection, survivorship and proxy biases. Secondly, most papers disregard the financial aspects of DEXs, trying to make predictions either on simple transaction statistics, elements of smart contracts, or token flow graph structures. While the researchers have shown that the feature set requires improvement (Wu et al., 2024; Yaremus et al., 2025; Wu et al., 2025; Lin et al., 2024; Ma et al., 2023), no research team has yet incorporated theoretically grounded financial measures beyond simple aggregates of prices, volumes, and transaction counts. Altogether, there is an evident gap in this area of research. This motivates development of a robust approach to problem statement in rug pull detection and exploration of richer, theory-based features.

2.2 Machine Learning in Financial Fraud Detection

There has been extensive research in the broader topic of ML financial fraud detection. The solutions proposed in these articles may be applied to rug pulls in order to improve the state-of-the-art models. Golmohammadi et al. (2014), as one of the earlier works, used simple OHLCV aggregates and a Naive Bayes model for the detection of price ma-

nipulation. However, later works employ more complex approaches to identify financial fraud.

The evident trend in newer articles is usage of more and more complex models and feature sets. For example, [Deng et al. \(2019\)](#) leveraged a multi-objective evolutionary algorithm on top of standard gradient boosting in order to make a practical detector of illegal insider trading. They also utilized the excess return and beta, financial ratios and corporate-governance indicators, such as H5 and Z indices, as part of their feature set. Similarly, [Deng et al. \(2021\)](#) extended their prior work on insider trading using several market-performance and governance measures. In [Wang et al. \(2019\)](#) the researchers used a recurrent neural network-based ensemble on market characteristic time series and static firm-level features to detect stock manipulation. [Liu et al. \(2021\)](#) enhanced their stock manipulation analysis with contextual features derived from analyst reports, financial news and social media posts. As an example from the general literature on cryptocurrency fraud, [La Morgia et al. \(2023\)](#) employed time-of-day features for real-time pump-and-dump scheme prediction in order to capture the well-documented intraday cyclicity in financial markets.

Additionally, in modern fraud detection work researchers often employ sophisticated tools to improve their models. As such, many studies used synthetic minority oversampling technique (SMOTE) to counter the inherent class imbalance in fraud detection problems ([Liu et al., 2021](#); [Hilal et al., 2022](#); [Srifa et al., 2025](#); [Kalacheva et al., 2025](#)). As an alternative solution to dataset limitations, recent papers have started to move away from supervised learning to unsupervised and semi-supervised learning, as well as generative models ([Hilal et al., 2022](#); [Mazzarisi et al., 2022](#)).

Overall, the research shows that informative, contextual, theoretically and empirically justified features are capable of enhancing the quality of fraud prediction ([Ashtiani and Raahemi, 2022](#)). Furthermore, purposeful feature engineering often contributes more to the detection efficiency than model selection ([Hilal et al., 2022](#)). At the same time, non-standard approaches to fraud detection, such as unsupervised and semi-supervised ML, may improve the resulting algorithm. This indicates a potential to improve the state-of-the-art rug pull detection models by introducing financial metrics as features or labeling criteria, as well as employing novel and non-standard model architectures. It particularly motivates exploring which piece of financial theory could best explain the patterns observed in rug pulls. All in all, it drew me to look into market microstructure theory as the solution to the feature set problem.

2.3 Market Microstructure Metrics as Predictors

Among the financial features that could enhance rug pull prediction, the ones seeming most appropriate come from the area called market microstructure—the study of trading

mechanics and frictions under explicit rules (O’Hara, 1997). While this section of financial theory was originally developed for classic financial markets, it is possible that the fundamental concepts are applicable to any form of trading, including DEXs.

One of the major topics in the market microstructure theory is information asymmetry and the resulting adverse selection. Arguably, the most important paper among not only the studies that tackle this specific issue, but also all works in market microstructure, is Kyle (1985). In this groundbreaking paper the author described a batch auction market model with a market maker, an insider and noise traders. The author finds a unique linear equilibrium, in which the insider hides their information by trading gradually, but eventually all information is incorporated into the price. An extremely important parameter, called Kyle’s lambda, stems from the equilibrium. This metric measures the price impact of order flow, the equilibrium liquidity, and the degree of informed trading activity. While Kyle’s lambda can capture various aspects of most financial markets, it is fundamental to assessing the information asymmetry. As rug pull tokens are assets with inherent asymmetry in the knowledge of the developers and investors, and the scam itself is the exploitation of that asymmetry, Kyle’s lambda should be used as a tool for rug pull detection. Easley-O’Hara’s PIN and VPIN are the other two widely used proxies for information asymmetry. In Easley et al. (1996) the authors described a sequential trade model, in which they calculated a theoretic probability of informed trading (PIN). The model gives an exact formula for the metric, which can be estimated as a structural measure from the buy-sell order flow. In Easley et al. (2012), the researchers developed a high frequency alternative to their metric—Volume-synchronized PIN (VPIN). It allows for a nonparametric measure of flow toxicity without structural estimation. Due to the theoretical connection of these measures to information asymmetry, they are highly suitable for inclusion in the rug pull detection models.

Another key theme in market microstructure literature is trading costs as a measure of liquidity, specifically estimation of the bid-ask spread. Several papers proposed ways to assess the difference in prices faced by traders depending on the trade direction without it being explicitly present in the available data. Roll (1984) described a simple market model in which he showed that the bid-ask spread can be estimated from the serial correlation in midquote changes, assuming the market is efficient. The key idea of the paper is to assume the existence of the efficient price, around which the midquote will fluctuate. This way Roll developed a measure of an implied spread, which can be calculated even without explicit bid and ask prices. Corwin and Schultz (2012) provided another spread estimate, which instead was based on the idea that high prices are produced by buy orders, and low prices – by sell orders. This way their spread estimate only requires high and low prices over several time bars, assuming that price variance scales linearly over time. In Goyenko et al. (2009) the authors assessed such liquidity estimates by comparing them to observable forms of spread. One of them is the effective spread, which measures

the discrepancy between the realized price of the trade with the preceding midquote. They show that low frequency spread estimates are capable of effectively estimating the high frequency spread measures, thus pointing to their capability of efficiently capturing liquidity dynamics, which is important for this study.

The last topic in market microstructure I will consider is price impact and friction. One of the most important papers on this topic is the aforementioned [Kyle \(1985\)](#), and various studies have been conducted to develop this work's fundamental ideas. As such, in [Kyle and Obizhaeva \(2016\)](#) the authors proposed the Invariance hypothesis, stating that the distributions of risk transfers and transaction costs are cross-sectionally constant when we measure them in business time. This way the researchers developed cross-sectional formulas for several quantities, including Kyle's lambda and transaction costs. These formulas provide a benchmark for the key market microstructure metrics, which could be used to detect abnormalities in the data. The other classic paper in this area is [Amihud \(2002\)](#), which introduced the standard measure of illiquidity, estimated as the average return on stock per dollar volume of the trade that produced it. The author also demonstrated the existence of an "illiquidity premium": higher illiquidity is correlated with higher expected returns. Several papers have investigated how order imbalance (OI), total buy volume net sell volume, impacts the price. [Chordia et al. \(2008\)](#) were able to demonstrate a strong positive impact of OI on next 5-minutes returns, though the dependency varied over time. [Cont et al. \(2014\)](#) went beyond the standard OI (they called it trade imbalance), which effectively only accounted for market orders, by including limit orders and cancellations into a so-called order flow imbalance (OFI). This way they showed a stable linear relationship between OFI and short-term returns on stocks, pointing out the potential connection to adverse selection effects.

While these market microstructure metrics are well-established in the financial literature, they have been used explicitly by only a handful of studies on decentralized exchanges (DEXs). On these new markets there is an abundance of granular data to utilize, yet no explicit measures of liquidity pool risk and quality – that is where market microstructure theory is very beneficial. As such, [Han et al. \(2024\)](#) used "Roll's lambda" (a modification of Kyle's lambda within Roll's model) and VPIN to demonstrate that a positive liquidity shock on Uniswap DEX converted to an increase in order flow toxicity on Binance centralized exchange (CEX). They also used Order Imbalance to show that traders on one platform respond to price shocks on the other exchange. [Alexander et al. \(2025\)](#) used (simplified) Kyle's lambda to show that liquidity has a mixed effect on different token pairs and liquidity pool architectures. [Aspris et al. \(2026\)](#) used Effective spread to analyze the gap between transactions costs on CEXs and DEXs, showing that the latter have more resilient liquidity in periods of volatility.

What we see with this new area of research is that quite often the well-established measures have to be adapted to capture the reality of the new financial markets. For

instance, [Zhu et al. \(2024\)](#) developed the Uniswap v3 effective spread, as well as the counterfactual Uniswap v2 spread to demonstrate how gas prices and volatility reduce liquidity. [Lehar and Parlour \(2025\)](#) used high-frequency analogue of Amihud’s measure, showing that price impact on Uniswap DEX is lower than on Binance CEX in periods of lower volatility and high noise trading.

To summarize, ML rug pull detection is a new area of research with several major papers establishing the fundamental aspects of these scams. Although several approaches to predicting this form of fraud have been proposed, many of the features they rely on lack financial justification and need significant improvement. Moreover, the labeling methodology requires robustness validation and potential refinement. As the research in other types of financial fraud detection demonstrates, a carefully formed set of informative, contextually appropriate features can provide significant improvement in the prediction quality. It is quite apparent that market microstructure metrics are capable of providing the needed improvement for the rug pull detection algorithms. Therefore, in my study the aim is to analyze the rug pull scams, develop a robust ex post identification methodology for this form of fraud and implement various market microstructure features in a ML rug pull detection, assessing their effect on the model’s performance.

3 Data

3.1 Scope of Research

As the goal of this study is to develop an early rug pull detection model, the Solana blockchain is the perfect environment to study them, as mentioned in the introduction. However, rather than analyze tokens launched across the entire blockchain, in this study I specifically focused on launches within the Raydium DEX. As we have seen in other papers in this area of research, different DEXs have significantly different patterns in how scams occur, making the detection of rug pulls more complex ([Yaremus et al., 2025](#)). For the purposes of this study, it is sufficient to concentrate on Raydium only, as in Q4 2024 it was responsible for around half of the total trading volume, while being the destination for newly graduated tokens from Pump.fun, the biggest platform for launching cryptocurrency ([Ilemi, 2026](#)).

I chose to study tokens launched in October and November 2024—the period in which the activity on Solana increased significantly, driven predominantly by a rise in popularity of Pump.fun and, therefore, elevated activity on the Raydium DEX ([Ilemi, 2026](#)). Specifically, this is the period in which the highest number of unique traders was observed, and the trading volume was getting close to its peak values. By choosing this period for analysis, I was able to study patterns in tokens both before the "meme coins supercycle" and around the time of its start, hopefully, capturing different regimes of the meme coins

market. I was specifically able to collect data for tokens with first trades from October 1 to November 12, 2024.

I analyzed tokens that apply to these criteria:

- Token's IDO (initial DEX offering) occurred between 2024-10-01 and 2024-11-12 on Solana's Raydium DEX.
- Around the time of IDO, there was no "fracturing" of trades, i.e. its only quote token was WSOL and there was no other liquidity pool/DEX it was traded within.
- Prior trading on Pump.fun is allowed, but the trading history on that platform is omitted.
- The only liquidity pool architectures allowed are the "standard" ones: AMMv4 and CPMM.

The last requirement is necessary for deterministic pricing of the token. The other more or less prevalent alternative architecture is CLMM, which allows liquidity providers to opt for a custom mechanism, making pricing ambiguous ([Raydium Protocol, 2025](#)).

The reason why I allow prior token history on Pump.fun (and only there) is that, among the platforms highly active at the time, it primarily served as a launch venue for tokens that later migrated to Raydium. Pump.fun uses a bonding-curve mechanism with a comparatively rigid and deterministic trading rule prior to graduation ([Pump Fun, 2026](#)). Therefore, while it is important to keep track of whether a token originates from Pump.fun, I do not model the pre-graduation bonding-curve dynamics in this thesis and focus on the Raydium trading phase.

Under these conditions, I was able to collect 32,426 tokens that had at least 30 transfers from or to liquidity pool's vaults within first 5 minutes of trading (apart from the liquidity pool deployment transaction). I set this requirement to filter out tokens with extremely low early activity, for which transaction aggregates and microstructure-style statistics are unreliable. In practice, tokens with near-zero early activity are also not relevant for short-horizon trading decisions.

3.2 Data Collection

In this study the main sources of data are Dune Analytics and the Raydium API. Dune provides convenient access to decoded historical on-chain data for Solana via SQL queries. The Raydium API is used to obtain static liquidity-pool attributes, such as program id, vault accounts and LP mint. The interaction with the API was done in Python.

The data collection consisted of 4 main steps:

1. Retrieve from Dune for each token-pool pair its first trade time and metadata, filter out based on requirements discussed in section [3.1](#).

2. Enrich the tokens list with static attributes of the liquidity pool via Raydium API.
3. For each token collect from Dune the transfers that 1) occurred within 5 minutes since the first trade time and 2) involved the token’s liquidity pool vault accounts. Recombine the transfers into atomic transactions via transaction id. Store vault deltas (net changes in vault account volumes) and recover the on-chain ordering of transactions so as to construct the event sequence for each token.
4. Similar to previous step, reconstruct vault deltas for the first 1 hour of trading, from which TAQ-like data is identified and target components are calculated, as discussed in section 4.

Importantly, in the 3rd and 4th steps I added a lookback of 10 minutes for transfer data, allowing any transaction (especially the pool liquidity deployment transaction) that happened before the first trade to be captured. The reason we need to find all history of transfers is that the transfers table only contains changes in accounts, not their states. The complete token account state can be found in a much larger transactions table, however with limited financial and computational budgets I had to forego exploring the transactions table directly. By setting such a lookback, we assume that for these tokens the trading is not delayed for more than 10 minutes. In practice, most new tokens are almost right away tracked and traded on by sniper bots, making such a lookback more than enough (Kalacheva et al., 2025). To make it significantly less likely that the pool creation is missed and ensure that the cumulative values are reliable, I filtered out tokens that did not have liquidity provision operation as the first transaction observed. More specific details on the data collection process can be found in the GitHub repository (Kliuchnikov, 2026).

4 Methodology

4.1 Problem Specification and Targets

As described in section 2.1, there is no unique robust labeling strategy for rug pulls. However, this does not stop us from trying to see whether market microstructure measures would be able to capture patterns in rug pull events. In many applied risk settings, the adverse event of interest is difficult to observe directly or define unambiguously. A common approach is therefore to study proxy outcomes that capture relevant failure types and can be constructed at scale.

I apply this perspective to the labeling problem: all the rug pull labeling strategies explored in previous work are valid proxy constructions, while none of them truly identify rug pulls. The set of problem specifications from preceding papers, therefore, constitutes a family of problems, aimed at specific types of rug pull-like events or, more generally,

tokens with an extreme early drop in liquidity. This is similar to the approach in [Yaremus et al. \(2025\)](#), but without any claim for true rug pull identification. Thus, I created a list of targets, based on the ones developed in previous work.

Let us consider the statistics that were used in past studies on rug pull detection to construct the targets. As in [Mazorra et al. \(2022\)](#), we define a maximum drop (simplified maximum drawdown) of series X_i with $i \in [1, N]$ as

$$MD_X = \frac{X_h - X_l}{X_h}, \quad (1)$$

where $X_h = \max_{i \in [1, N]} X_i$, $h = \operatorname{argmax}_{i \in [1, N]} X_i$, $X_l = \min_{i \in [h, N]} X_i$. Effectively, this is the maximum drawdown under the assumption that the largest drop happens off the peak value in X_i , which is, in general, not true. Nonetheless, this is the approach proposed in Mazorra et al. (2022), which has been adapted in [Srifa et al. \(2025\)](#) and [Yaremus et al. \(2025\)](#), so I will not alter it. Importantly, in Srifa et al. (2025) specifically authors do not require X_l to be calculated as the minimum after the peak, but rather the lowest point of X_i in general. This is, obviously, incorrect to use—otherwise a long rise in value of an asset may be considered a drop in value instead. That is why I assume that their intention was to replicate the approach from Mazorra et al. (2022) and the formula in their paper is incorrect, while the actual study likely used the correct estimation.

In addition to an extreme maximum drop, Mazorra et al. (2022) require that there is almost no recovery in value. For a series X_i with $i \in [1, N]$, its recovery is calculated as

$$RC_X = \frac{X_N - X_l}{X_h - X_l}, \quad (2)$$

where X_l and X_h are defined as before. Effectively, it estimates the fraction of value recovered from drop to the end of observation period.

The last statistic we consider is longest inactivity, proposed in [Kalacheva et al. \(2025\)](#). For transaction timestamps $0 = t_1 \leq t_2 \leq \dots \leq t_N \leq T$, the longest inactivity interval is defined as

$$\delta T = \max\left(\max_{i \in [2, N]} (t_i - t_{i-1}); T - t_N\right), \quad (3)$$

where δT is calculated in seconds.

Using these, I defined and calculated several targets (MD_p stands for maximum drop in price, MD_L – for liquidity; similarly for RC):

- `md_price_no_post_rc_0.9_0.01`: indicator of $MD_p \geq 90\%$ & $RC_p \leq 1\%$, Mazorra et al. (2022)-style price rug pull-like event
- `md_price_no_post_rc_0.99_0.005`: indicator of $MD_p \geq 99\%$ & $RC_p \leq 0.5\%$, Srifa et al. (2025)-style "hard" rug pull-like event

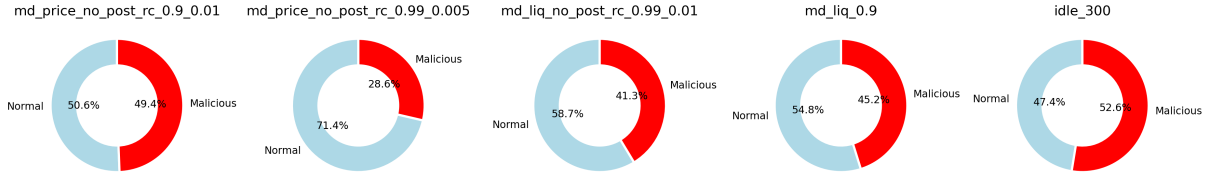


Figure 1: Distributions of calculated targets.

- md_liq_no_post_rc_0.99_0.01: indicator of $MD_L \geq 99\%$ & $RC_L \leq 1\%$, Mazorra et al. (2022) and Srifa et al. (2025)-style liquidity rug pull-like event
- md_liq_0.9: indicator of $MD_L \geq 90\%$, effectively the "TVL approach" from Yaremus et al. (2025)
- idle_300: indicator of $\delta T \geq 5min.$, "Idle approach" from Kalacheva et al. (2025) and Yaremus et al. (2025)

These targets are not entirely the same as in the original papers. First of all, most of past studies do not provide exact thresholds that they used in their analysis. For instance, in Mazorra et al. (2022) they do not specify how they detect that the token's price or liquidity had "almost no recovery". In most cases specific thresholds had to be chosen that overall match the seemingly intended specifications, based purely on phrasing.

Secondly, in these papers (which is also not always directly stated) "liquidity" is calculated as dollar TVL (total value locked) in the liquidity pool, which is simply the dollar value of the quote tokens in vault accounts. Because the observation window is so small, the price of the quote token (WSOL, in our case) stays overall the same, while the statistics are relative measures. That means that we can replace dollar TVL with WSOL TVL, i.e. the amount of WSOL stored in the quote vault, as the "liquidity" estimate.

Third, these targets were used a bit differently from the way I used them. In Mazorra et al. (2022) and Srifa et al. (2025) they additionally required the token to be active long term in order to be considered non-malicious. However, in our case, as explained next, the observation window is small, and there is inherently no way and, more importantly, no purpose to track long-term activity. Additionally, Mazorra et al. (2022) do not use the label I provided to identify "normal" tokens—that is done via audit data. Again, it would not only be unfeasible for that amount of tokens I am considering to collect audit data for all of them without selection bias, but it also is, in most cases, not important in context of short-term dynamics of a token.

In Figure 1 we can see what the resulting target distributions look like on the collected data. Importantly, the class distributions are not as imbalanced as in previous papers, in some of which the fraction of tokens that were labeled malicious was above 90%.

I will specifically focus on early rug pull detection. While several papers explored long term dynamics of token trading, Srifa et al. (2025) found that rug pull prediction can be

done in very short observation windows. Following this approach, Yaremus et al. (2025) tried predicting whether a rug pull will occur within the first hour of trading using data collected in first 5 minutes of token's existence. As I have explained previously, while this may not be a direct detection of a rug pull, such scams occur very quickly, as documented by several studies. The token creators who want to do a rug pull want to withdraw their liquidity as soon as they see the hype around the launch start diminishing. As traders are incentivised to invest as early as possible, the actual lifecycle of the token often becomes very short. In this study I used the same approach as in Yaremus et al. (2025), to, effectively, answer this question: "Within the first 1 hour since first trade of the token, will an adverse event X happen?", where X is one of the rug pull-like events described with targets above. The resulting early rug pull-like event detection models may become a risk assessment tool for short-term investment into new tokens. Additionally, I set the "launch time" to be the first trade timestamp for that token, rather than its initial mint or pool deployment. This is the approach most consistent with previous literature.

4.2 Data Engineering

In order to construct the features, I had to reproduce TAQ-like data (granular transaction data). I specifically needed to track the midquote dynamics, transaction type, trade sign in swaps, effective price paid and per-trade base and quote volume. I would like to note that from here on the "amount" I use in calculations is not raw, i.e. the actual number of individual tokens. Each token has so-called "decimals", which describe how the raw amount has to be re-scaled to be written in "human" terms:

$$amount = amount_raw \times 10^{-d}, \quad (4)$$

where d is the token's decimals; $d = 9$ for WSOL. Effectively, this allows to trade fractional amounts of tokens with a large integer number of them existing in reality. So, it is more appropriate to use the corrected amounts rather than raw, as intended.

As it is standard in AMM (automated market maker) markets, which include AMMv4 and CPMM architectures on Raydium, the midquote is found as

$$p_t = \frac{y_t}{x_t}, \quad (5)$$

where y is the amount of quote token (WSOL) in the liquidity pool quote vault and x is the amount of base token (the newly launched token) in the base vault. As there is no accumulated vault account amounts reported in the transfers table, we have to calculate

x_t and y_t from per-transaction "vault deltas" Δx_t and Δy_t , as discussed in section 3.2:

$$x_t = \sum_{i=0}^t \Delta x_i, \quad y_t = \sum_{i=0}^t \Delta y_i \quad (6)$$

This is possible due to the fact that the addition of liquidity to the liquidity pool at its deployment, as well as all changes in vault account storage, are tracked via transfers, each being part of a specific transaction. Therefore, the entire liquidity dynamic can be reproduced from individual transfers. At the same time, all components of TAQ-like data are reproducible from the vault deltas. Specifically, swaps are identifiable by the fact that the corresponding vault deltas are of opposite sign (bought one token, paid another), the effective trade price is the ratio of the deltas, and the trade sign b_t is simply the sign of quote vault delta:

$$is_swap = \mathbb{I}\{\Delta x_t \cdot \Delta y_t < 0\} \quad (7)$$

$$p_t^{eff} = \left| \frac{\Delta y_t}{\Delta x_t} \right| \quad \text{if } is_swap \quad (8)$$

$$b_t = \text{sign}(\Delta y_t) = -\text{sign}(\Delta x_t) \quad \text{if } is_swap \quad (9)$$

Additionally, transactions that have both deltas positive are labeled lp addition-like (transactions that have the same affect on the pool's liquidity as liquidity provision (lp mint)), the ones with both negative are lp removal-like (similar to lp burn transactions). Decoding transfer events into these time series is sufficient to construct almost all features.

4.3 Features

4.3.1 Benchmark

The aim of the study is to conduct an A/B test on two models—benchmark and expanded—in order to identify whether theoretically justified market microstructure measures have properties that translate beyond standard financial markets to decentralized finance and are capable of improving rug pull detection. Therefore, we have to specify what features are included into the benchmark and expanded models.

The complete list of benchmark features can be found in Appendix A. This list was my attempt to combine all features that have been used in previous studies. While a lot of them are quite simple and self-explanatory, there is one important point to mention. Like with targets, the authors in some cases did not provide specific details on how a certain feature is calculated. For instance, it is not specified whether the buy percentage is calculated against all transactions or swaps only. The table in the appendix describes an overall idea of how the feature was estimated. For specific details (for example, Relative Strength Index implementation), visit the Github repository ([Kliuchnikov, 2026](#)).

I was not able to reconstruct all features from past research, though in most cases they would not be applicable or useful in this particular study. Specifically, I excluded HHI of token ownership and clustering coefficient (Mazorra et al., 2022), as that would require a complex analysis of the entire transfer graph of the token, which can be enormous for a modern cryptocurrency. We have seen that these features either way did not have high importance in the model’s predictions, so we may assume that omitting them is not a significant problem. I did not calculate validator bribes or sniper-related features (Kalacheva et al., 2025), as they are quite specific to the Ethereum blockchain and not applicable to Solana per se. I also did not utilise smart contract code features, as, in reality, most tokens are launched in a similar way, and a lot of them start on Pump.fun, making smart contract code unlikely to be useful for rug pull detection, not to mention the complexity of data mining required to retrieve such data. Finally, I did not use features that calculated value in dollars, but rather used WSOL instead. While WSOL is a valid measure of grounded token value, it is possible that the underlying dynamic in the value of WSOL to dollars should be accounted for, which may be explored in future research.

I also added "static" features (fixed since the moment of IDO) to both benchmark and expanded features sets. These are pool type (2.0% are CPMM, 98.0% are AMMv4) and decimals. These features are important to have in order to account for architecture-specific patterns. It would also be useful to add an indicator of graduation from Pump.fun into the feature set. However, as displayed in SHAP values (Appendix B), the feature `has_pumpdotfun_history`, indicator of whether Dune Analytics collected Pump.fun trades for that token, is worryingly effective at identifying liquidity rug pulls. Moreover, while Pump.fun is normally considered a place of origin for most rug pulls, SHAP values demonstrate that presence of Pump.fun history decreases the probability of a rug pull. This may indicate survivorship bias in Dune’s processing of Pump.fun trades. While there is little doubt in the quality of Raydium data on Dune, I am quite concerned that `has_pumpdotfun_history` could create target leakage in the feature set. For that reason I excluded it. It is, thus, an aim for future research to create a robust identification process for tokens that graduated from that platform using the instructions table.

4.3.2 Market Microstructure Measures

Most importantly, I extended the benchmark feature set with new features that come from the market microstructure theory. Specifically, I calculated

- Normalized Kyle’s lambda: following Kyle (1985) and Lopez de Prado (2018), I calculated Kyle’s lambda as λ in the linear regression

$$\Delta p_t = c + \lambda b_t V_t + \epsilon_t, \quad (10)$$

where Δp_t is the change in midquote, b_t is the trade sign (buy = 1, sell = -1), and

V_t is the volume (number of tokens traded), hence $(b_t V_t)$ is the signed volume.

However, comparing Kyle's lambda across different tokens is questionable. The metric measures absolute price impact per unit of signed volume traded, but the price levels and representative per-trade volume differ across tokens. As a result, as data shows, Kyle's lambda has an extremely high (above 0.9) Spearman correlation with maximum price observed and relatively high absolute Spearman correlation with minimum value traded.

An alternative approach can be found in [Kyle and Obizhaeva \(2016\)](#). There researchers, as a part of their cross-sectional microstructure analysis, propose to study the impact of each "bet" relative to the current price:

$$\tilde{\lambda}_t := \lambda_t \frac{V_t}{p_t}, \quad (11)$$

where λ_t is the current (theoretical) Kyle's lambda. Thus, replacing λ_t with λ estimated via the regression (10), we can calculate a normalized Kyle's lambda—average price impact of a trade relative to the preceding midquote—as

$$\tilde{\lambda} := \lambda \cdot \hat{\mathbb{E}} \left[\frac{V_t}{p_t} \right] \quad (12)$$

- [Harris \(1990\)](#) version of Roll's spread estimate ([Roll, 1984](#)): as there is a high chance that the differenced price series of a new token has a positive serial correlation (e.g., due to growing traction), I decided to implement the variation of Roll's spread estimate that works in this situation. The idea was that, while the actual bid-ask spread can only be positive, negative values of Roll's measure could indicate more information, beyond what is known in the standard financial literature. The Harris's modification is

$$s^{Roll} = \begin{cases} 2\sqrt{-Scov}, & Scov \leq 0 \\ -2\sqrt{Scov}, & Scov > 0 \end{cases}, \quad Scov = Cov(\Delta p_t^{eff}, \Delta p_{t-1}^{eff}) \quad (13)$$

(It is crucial that we use the effective price of the trade and not the midquote, as assumed in the model.)

However, like with Kyle's lambda, the raw spread level has high correlation with the price level and not appropriately comparable across tokens. That is why I applied the normalization that was actually used by Roll in the original paper—Roll's "percentage" spread—with the same Harris's modification:

$$s_{\%}^{Roll} = \begin{cases} 200\sqrt{-Scov_{\%}}, & Scov_{\%} \leq 0 \\ -200\sqrt{Scov_{\%}}, & Scov_{\%} > 0 \end{cases}, \quad Scov_{\%} = Cov(R_t, R_{t-1}), \quad (14)$$

where R_t represents token's return (percent difference between consecutive trade prices).

- Amihud's illiquidity measure, following [Amihud \(2002\)](#) and [Hasbrouck \(2009\)](#), as

$$I = \frac{1}{T} \sum_{t=1}^T \frac{|R_t|}{|p_t V_t|}, \quad (15)$$

where T is the number of trades, and $p_t V_t$ is the WSOL volume swapped.

- A variation of VPIN, based on [Easley et al. \(2012\)](#) and [Lopez de Prado \(2018\)](#), as

$$VPIN = \frac{1}{n} \sum_{\tau=1}^n \frac{|V_{\tau}^B - V_{\tau}^S|}{V}, \quad (16)$$

where V is the trade volume in each volume bar $\tau \in [1, n]$, and V_{τ}^B and V_{τ}^S are buy and sell volumes within those volume bars, respectively.

I had to make a few changes to how VPIN is calculated to adapt it to early token life analysis. In the original paper ([Easley et al. \(2012\)](#)) authors use past trading data to find the appropriate volume bucket size V . However, new tokens do not have such history, so I used the first 5 minute data to choose the volume bin size before calculating VPIN. Specifically, I calculated it to be such that we can expect 10 volume bars if the mean trade volume is close to median size. By using median as the measure of representative trade volume, I simulated the approach from the original VPIN paper, as outlier trade volumes, which median does not capture, are effectively unexpectedly large/small, and choice of bucket size based on historic observations would similarly not account for an arrival of a trade of such size. Though we technically select parameters on the data that we want to calculate on next, this does not cause target leakage or any other form of bias. Additionally, the authors proposed using batch classification of trades, rather than use the standard tick rule, but that also requires past data. I instead classify swap direction and corresponding volume using the sign of the quote vault (WSOL) delta.

- Order Imbalance (OI): like in [Chordia et al. \(2008\)](#) and [Cont et al. \(2014\)](#), calculated as

$$OI = \sum_{t=1}^T b_t V_t \quad (17)$$

- Median effective spread: as presented in [Goyenko et al. \(2009\)](#), one can estimate

the effective spread as

$$s_t^{eff} = 2 \cdot \left| \log p_t^{eff} - \log p_{t-1} \right|, \quad (18)$$

This way we can construct a time series of effective spreads. One way to aggregate the series into a feature is to use median s_t^{eff} as an estimate of the effective spread faced in a representative trade.

- Corwin-Schultz spread estimate: based on [Corwin and Schultz \(2012\)](#) and [Lopez de Prado \(2018\)](#), one can estimate the spread via high and low prices, as

$$s_t^{CS} = \frac{2(e^{\alpha_t} - 1)}{1 + e^{\alpha_t}}, \quad (19)$$

where

$$\alpha_t = \frac{\sqrt{2} - 1}{3 - 2\sqrt{2}} \cdot \sqrt{\beta_t} - \frac{1}{\sqrt{3 - 2\sqrt{2}}} \cdot \sqrt{\gamma_t}, \quad (20)$$

$$\beta_t = \mathbb{E} \left[\left(\log \frac{H_t}{L_t} \right)^2 + \left(\log \frac{H_{t-1}}{L_{t-1}} \right)^2 \right], \quad (21)$$

$$\gamma_t = \left[\log \left(\frac{H_{t-1,t}}{L_{t-1,t}} \right) \right]^2, \quad (22)$$

where H_t and L_t are high and low prices, respectively, within time bar t , $H_{t-1,t}$ and $L_{t-1,t}$ are high and low prices, respectively, across both time bars t and $t - 1$. The expectation in the formula for β_t is normally taken over a rolling window.

I implemented this formula using 30 seconds time bars, with β_t calculated as the expected value across $t \in [2, T]$. Additionally, while [Corwin and Schultz \(2012\)](#) recommend clipping negative α_t to 0 (equivalent to cutting off the spread at 0), I kept the negative spread due to the unique nature of the studied market; the negative values are likely to carry additional information a model could utilize. Overall, the justification behind allowing negative Corwin-Schultz spread values is the same as behind applying Harris’s modification to Roll’s measure.

Table 1 reports the descriptive statistics for the market microstructure features, as well as some of the benchmark features. As one can see, some of the features are highly skewed, potentially identifying anomalous values that the model can utilize. Roll’s spread is missing for observations with insufficient price variation or minimum price near 0.

4.4 Identification of Model Improvement

In order to see whether the dataset extension improves the model performance, I trained and evaluated a model on both the benchmark dataset and the expanded version. To do

Variable	Count	Mean	Std. Dev.	5%	Median (50%)	95%
Market Microstructure Features						
Norm. Kyle’s Lambda	32,426	1.48×10^9	2.65×10^{11}	4.57×10^{-4}	0.021	0.273
Roll’s % Spread	28,337	7.26×10^6	3.18×10^8	-248.84	-5.461	0.511
Amihud’s Illiquidity	32,426	9.48×10^{11}	1.62×10^{14}	2.58×10^3	2.15×10^4	3.09×10^8
VPIN	32,426	0.653	0.322	0.116	0.700	1.000
Order Imbalance	32,426	5.43×10^{12}	5.39×10^{14}	-4.16×10^8	5.32×10^7	8.21×10^9
Median Effective Spread	32,426	0.185	1.143	0.005	0.012	0.081
Corwin-Schultz Spread	32,426	-0.624	1.486	-2.000	-1.269	1.998
Selected Benchmark Features						
Number of Swaps	32,426	475.32	446.14	42.00	408.00	1338.75
Buy-Sell Ratio	32,373	3.31	6.69	0.67	1.63	11.00
Initial TVL (WSOL)	32,426	128.76	206.89	0.50	79.01	700.17
WSOL Turnover	32,426	762.68	1052.55	5.97	462.75	2364.50
Return (5 min)	32,426	7.79×10^8	1.39×10^{11}	-0.90	0.49	5.45×10^5

Table 1: Descriptive statistics of the key variables

so, I split the dataset into train and test by date: train consists of tokens launched from 2024-10-01 to 2024-10-31 (22611 samples), test has launches between 2024-11-01 and 2024-11-12 (9815 samples). After that I used evaluation metrics to compare the models before and after adding the features. The approach is robust to model choice and hyperparameter tuning. I used SHAP values to evaluate and interpret the contribution of each market microstructure metric to the model’s prediction (Lundberg and Lee, 2017). That is done in order to verify that the change in model performance is due to the new features, and also to understand how they affect the estimated probability of a rug pull-proxy event and see whether that is consistent with theory or not.

I chose to use 3 performance metrics: ROC AUC, as it is the industry standard and the easiest to interpret, PR AUC, as it is more robust to class imbalances, and F1 score, as it robustly evaluates predictions with respect to class 1, which we are more interested in (Saito and Rehmsmeier, 2015). To see whether the difference in metrics is statistically significant, I used 2 methods. First, as ROC AUC is equivalent to Mann-Whitney U statistic, there is a nonparametric statistical test applicable specifically to it, called fast DeLong’s algorithm (Sun and Xu, 2014). Second, I used the standard paired bootstrap approach to estimate the p-value for the changes in PR AUC and F1 score, as it is applicable to any performance measure (Efron and Tibshirani, 1994).

I applied both a gradient boosting model (CatBoostClassifier) and a random forest model (sklearn’s RandomForestClassifier) for robustness to model choice, as these are the options that are predominantly used in tabular data machine learning (Breiman, 2001; Prokhorenkova et al., 2018). I report the results for both the models with default specifications and the models to which I applied hyperparameter tuning via Optuna (Akiba et al., 2019). Tuning was done on walk-forward cross validation (TimeSeriesSplit in sklearn) due to the financial nature of the data: there may be significant fixed time effects that

can influence the rug pull patterns, which would cause information leakage in standard cross validation. The hyperparameters tuned and implementation details can be found in Appendix C.

5 Results

5.1 Change in Model Performance

Table 2 showcases the effect of extending the dataset with market microstructure features on models with default hyperparameters. As we can see for CatBoostClassifier, only on 2 proxy problems the expansion had a statistically significant effect on model's performance in at least one performance metric. Meanwhile, RandomForest classifier's predictions improved on 4 out of 5 problems, with clearly lower p-values. Overall, at least one model's performance improved on each of the rug pull proxy problems. Additionally, we see that on almost all of the tasks CatBoostClassifier had higher performance than RandomForestClassifier, with the only exception of benchmark F1 score when predicting idle activity.

The improvements demonstrate that market microstructure metrics are informative. However, the effect of the feature set expansion is inconsistent between models and targets. There is a well-known reason why that may happen – a more complex dataset may not be processed correctly by a model that was not correctly adapted to it. A model without tuning can easily overfit on the training set, especially considering that the number of features in the dataset is so large. That is why the comparison may be invalid, unless we first apply hyperparameters tuning. Therefore, I used Optuna with walk-forward cross-validation in order to conduct the same form of A/B test on tuned models.

In Table 3 we can see that the tuned models are more effective across all tasks, but also that the dataset extension improved models' performance on a majority of the targets. Similar to what can be seen in the default setting, a significant improvement is made in at least one of the models for each target, and for 2 of the targets (1 and 4) both models had higher performance. We also see model-specific improvements: ROC AUC and PR AUC significantly increased for the gradient boosting model on the "extreme TVL drop" and "Idle approach" tasks, while the random forest classifier had higher scores on the "extreme price drop" task. Also, interesting to note that whenever there is improvement in ROC AUC, PR AUC is also significantly different, and vice versa.

Like before, CatBoostClassifier outperforms RandomForestClassifier on all targets except the 5th, where again the benchmark F1 score is lower. Interestingly, that is also where the tuned random forest had nearly significant (at 90% confidence level) drop in F1 measure—the only decrease in performance in the A/B test. As I discuss in Appendix D.3, there are reasons to question the correctness of F1 score, which also serve as ways to

Target	Model	ROC AUC	PR AUC	F1 Score
CatBoostClassifier (default)				
$MD_p \geq 90\%$ & $RC_p \leq 1\%$	Benchmark	0.8700	0.8768	0.7825
	Expanded (<i>p-value</i>)	0.8721 (0.0161) *	0.8785 (0.0531) .	0.7851 (0.2807)
$MD_p \geq 99\%$ & $RC_p \leq 0.5\%$	Benchmark	0.9430	0.8713	0.7980
	Expanded (<i>p-value</i>)	0.9436 (0.3663)	0.8723 (0.4683)	0.7953 (0.4082)
$MD_L \geq 99\%$ & $RC_L \leq 1\%$	Benchmark	0.9690	0.9465	0.8689
	Expanded (<i>p-value</i>)	0.9696 (0.1973)	0.9488 (0.0078) **	0.8671 (0.4184)
$MD_L \geq 90\%$	Benchmark	0.9611	0.9466	0.8638
	Expanded (<i>p-value</i>)	0.9619 (0.1438)	0.9481 (0.0536) .	0.8619 (0.3695)
$\delta T \geq 5min.$	Benchmark	0.8985	0.9034	0.8092
	Expanded (<i>p-value</i>)	0.8998 (0.1115)	0.9047 (0.1278)	0.8129 (0.0812) .
RandomForestClassifier (default)				
$MD_p \geq 90\%$ & $RC_p \leq 1\%$	Benchmark	0.8613	0.8650	0.7802
	Expanded (<i>p-value</i>)	0.8649 (0.0022) **	0.8697 (0.0006) ***	0.7832 (0.2509)
$MD_p \geq 99\%$ & $RC_p \leq 0.5\%$	Benchmark	0.9401	0.8587	0.7810
	Expanded (<i>p-value</i>)	0.9425 (0.0058) **	0.8650 (0.0003) ***	0.7899 (0.0164) *
$MD_L \geq 99\%$ & $RC_L \leq 1\%$	Benchmark	0.9678	0.9401	0.8687
	Expanded (<i>p-value</i>)	0.9684 (0.2115)	0.9416 (0.1983)	0.8649 (0.1166)
$MD_L \geq 90\%$	Benchmark	0.9586	0.9402	0.8587
	Expanded (<i>p-value</i>)	0.9602 (0.0048) **	0.9426 (0.0155) *	0.8583 (0.8714)
$\delta T \geq 5min.$	Benchmark	0.8921	0.8946	0.8105
	Expanded (<i>p-value</i>)	0.8951 (0.0014) **	0.8988 (0.0014) **	0.8117 (0.6007)

Table 2: Change in performance metrics of default CatBoostClassifier (top) and RandomForestClassifier (bottom). Significance codes:

. $p \in [0.05, 0.1]$; * $p \in [0.01, 0.05]$; ** $p \in [0.001, 0.01]$; *** $p < 0.001$.

Target	Model	ROC AUC	PR AUC	F1 Score
CatBoostClassifier (tuned)				
$MD_p \geq 90\%$ & $RC_p \leq 1\%$	Benchmark	0.8706	0.8773	0.7847
	Expanded (<i>p-value</i>)	0.8725 (0.0125) *	0.8794 (0.0093) **	0.7866 (0.3912)
$MD_p \geq 99\%$ & $RC_p \leq 0.5\%$	Benchmark	0.9443	0.8737	0.7963
	Expanded (<i>p-value</i>)	0.9449 (0.1907)	0.8742 (0.6602)	0.7944 (0.4350)
$MD_L \geq 99\%$ & $RC_L \leq 1\%$	Benchmark	0.9708	0.9499	0.8715
	Expanded (<i>p-value</i>)	0.9716 (0.0048) **	0.9515 (0.0092) **	0.8729 (0.3578)
$MD_L \geq 90\%$	Benchmark	0.9618	0.9474	0.8662
	Expanded (<i>p-value</i>)	0.9630 (0.0034) **	0.9496 (0.0002) ***	0.8641 (0.2858)
$\delta T \geq 5min.$	Benchmark	0.8989	0.9045	0.8121
	Expanded (<i>p-value</i>)	0.9012 (0.0008) ***	0.9065 (0.0126) *	0.8130 (0.6166)
RandomForestClassifier (tuned)				
$MD_p \geq 90\%$ & $RC_p \leq 1\%$	Benchmark	0.8637	0.8718	0.7766
	Expanded (<i>p-value</i>)	0.8662 (0.0099) **	0.8744 (0.0034) **	0.7781 (0.5849)
$MD_p \geq 99\%$ & $RC_p \leq 0.5\%$	Benchmark	0.9425	0.8674	0.7852
	Expanded (<i>p-value</i>)	0.9447 (1.1e-05) ***	0.8712 (0.0002) ***	0.7875 (0.4260)
$MD_L \geq 99\%$ & $RC_L \leq 1\%$	Benchmark	0.9708	0.9487	0.8714
	Expanded (<i>p-value</i>)	0.9706 (0.4814)	0.9478 (0.2201)	0.8728 (0.4194)
$MD_L \geq 90\%$	Benchmark	0.9603	0.9445	0.8623
	Expanded (<i>p-value</i>)	0.9619 (0.0004) ***	0.9476 (0.0000) ***	0.8638 (0.4877)
$\delta T \geq 5min.$	Benchmark	0.8970	0.9017	0.8151
	Expanded (<i>p-value</i>)	0.8972 (0.8785)	0.9031 (0.1791)	<u>0.8114</u> (0.0903) .

Table 3: Change in performance metrics of CatBoostClassifier (top) and RandomForestClassifier (bottom) with tuned hyperparameters. Significance codes:
. $p \in [0.05, 0.1]$; * $p \in [0.01, 0.05]$; ** $p \in [0.001, 0.01]$; *** $p < 0.001$.

interpret lack of significant change identified for this metric.

Overall, the change in performance metrics is statistically significant and there is no doubt about the existence of additional information captured by the new features. However, when considering the absolute change in performance metrics, we do not see such a drastic improvement, as the differences are mostly less than one percentage point. This may be caused by the fact that, as we see in the benchmark metrics, the models are already highly efficient for these tasks. It is well established that while absolute changes in performance metrics may be marginal, when deployed at scale in algorithmic trading and risk management system, the monetary value of such improvements can be very large. However, this may also be indicative of need for other market microstructure measures, or other improvement in the model setup. Overall, further improvement of the model is definitely still an open and important direction for future research.

These results also demonstrate an important point. While standard models may have only marginal gains, hyperparameter tuning unlocks the predictive power of microstructure features, as seen for CatBoostClassifier. This suggests that these features are complex and non-linear, requiring optimized model architecture to be fully utilized, as supported by [Lopez de Prado \(2018\)](#).

5.2 Interpretation

Let us look at specifically the first target, [Mazorra et al. \(2022\)](#)-style price rug pull proxy—effectively the key benchmark in this area of research—and see where the improvement of tuned models comes from. Figure 2 demonstrates the most important features in terms of SHAP values and their influence on the estimate of rug pull probability. We see market microstructure metrics appear for both models. Specifically, order imbalance measure appears at the top for both of them, while median effective spread is highly important for CatBoostClassifier, and Roll’s percentage spread and VPIN are shown to have high importance in RandomForest model’s prediction. Overall, it indicates that market microstructure features were important in those predictions, and the importance of OI translates between models. This makes sense, as this particular proxy corresponds to a drop in price soon after launch, and specifically OI has been shown to be an effective predictor of short-term returns ([Chordia et al., 2008](#); [Cont et al., 2014](#)).

Let us also consider SHAP scatter plots for market microstructure metrics on that same target. Figure 3 shows that, overall, the SHAP values dependency on market microstructure measure’s value looks similar for both types of models, despite differing rankings among features. It is important to note that RandomForestClassifier’s plot look more "scattered", but this is mostly due to the SHAP values being approximate. With higher complexity (deeper trees with a comparable number of estimators) in the random forest compared to Catboost, the only reasonable way to calculate RandomForest’s SHAP

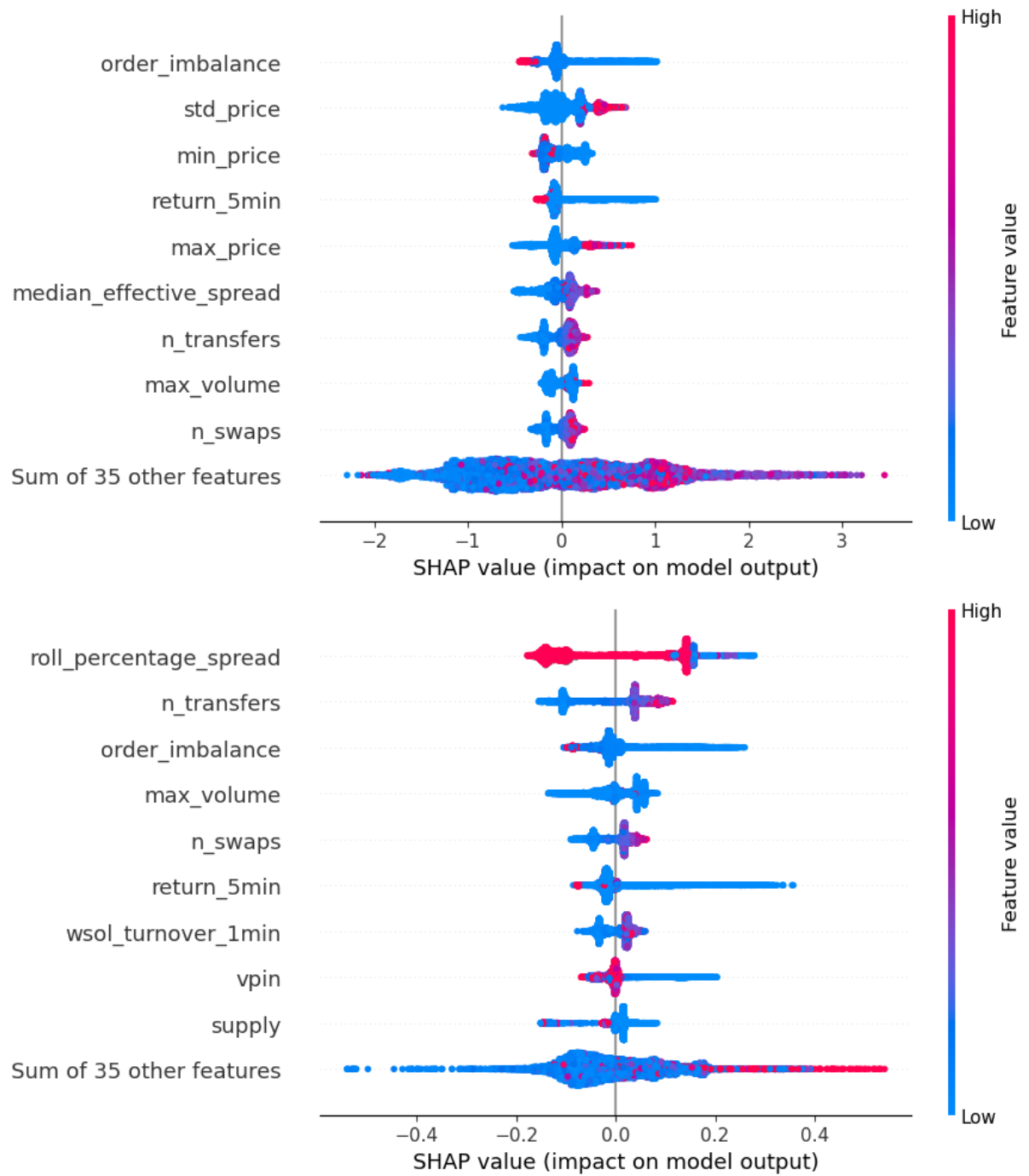


Figure 2: SHAP values beeswarm plot for tuned CatBoostClassifier (top) and RandomForestClassifier (bottom) models.

values is to use the approximation. However, that may cause some information about the model’s usage of the feature to be lost. That is why, for further analysis of SHAP scatter plots and their correspondence to market microstructure theory, I will only consider the output for Catboost.

Among these metrics, several of them impact the model’s prediction in the exact manner as what we would expect. For example, as normalized Kyle’s lambda increases, the estimated probability of the proxy event goes up. Specifically, extremely large values of lambda cause a very large increase in the estimated probability. This is consistent with theory: as lambda increases, it means that price can move more per given unit of volume, which means it is easier to manipulate the price and the token’s price is more vulnerable to extreme drops—exactly the proxy we are analyzing. Overall, we can say that on this dataset Kyle’s lambda operates as the detector of thin/abnormally low liquidity. The interpretation for Amihud’s illiquidity and median effective spread is very similar and even easier, as they are direct measures of illiquidity that may allow this type of proxy event to occur.

Order imbalance also has effect consistent with literature. As [Chordia et al. \(2008\)](#) and [Cont et al. \(2014\)](#) demonstrate, OI is highly (positively) correlated with short-term returns. Therefore, when OI is negative with high magnitude, we would expect a very low return, which corresponds to a significant drop in price—that is what the model seems to utilize. Similarly, high positive OI is correlated with high returns, which means that a significant drop is less likely, though still possible.

The interpretation of the scatter plots starts to diverge from market microstructure theory when we look at the features that had the most significant modifications. Specifically, Roll’s and Corwin and Schultz’s spread estimates, which normally cannot be negative, are actually mostly negative on this dataset after applying the modifications that allow them to be such. Moreover, the model seems to detect higher probability of a rug pull proxy event for larger in magnitude negative spread values and lower probability for positive and near-zero values.

How can we interpret the negative spreads? Let us first consider Roll’s spread. Under the assumptions of the model—that is, the price should fluctuate around the "efficient" level, the serial covariance of price changes, observed over a sufficiently large period, has to be negative in order to achieve such mean reversion ([Roll, 1984](#)). Therefore, under the model’s assumptions no negative values can appear under the square root. However, it is possible that in real data the serial covariance of price changes is positive over our observation window. If there is a more or less consistent movement of the price in one direction or another, the [Harris \(1990\)](#) version of Roll’s spread will be negative. In other words, when there is strong momentum in the price dynamic, such as a rapid price descent, or a price hike, also known as a pump, the spread estimate becomes negative. Therefore, the estimate’s sign is the market regime indicator that the model utilizes.

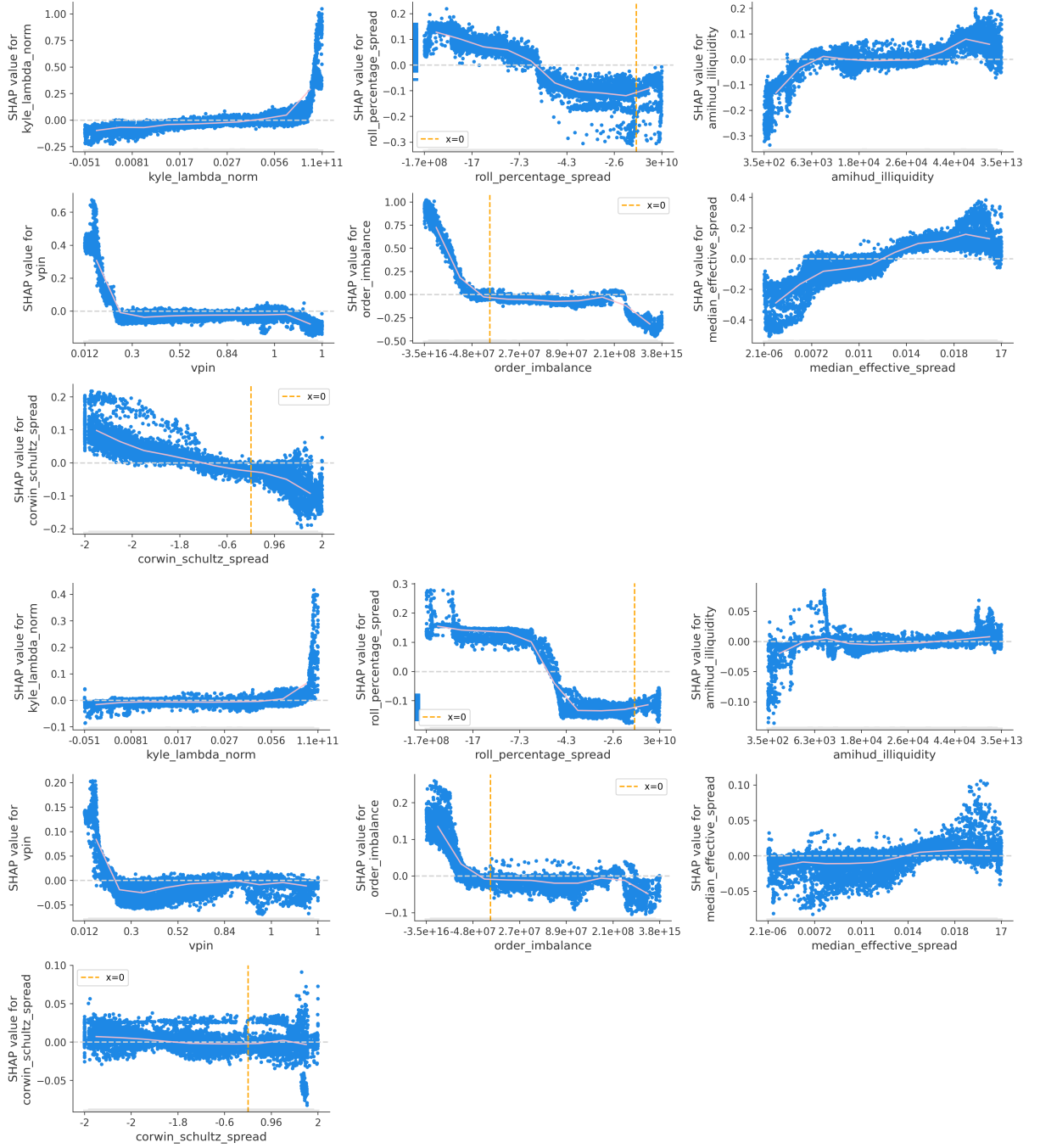


Figure 3: SHAP values scatter plots for tuned CatBoostClassifier (top) and RandomForestClassifier (bottom) models on market microstructure features. The pink lines show mean SHAP values within 10 quantile bins.

A very similar conclusion can be made about the Corwin-Schultz estimate. One of the key assumptions behind the measure is that variance scales linearly over time. However, as the authors noted themselves, if there is a period of high volatility, which corresponds to disproportionately greater price range in one of the time bins, the spread estimate can become negative. According to the formula presented in [Lopez de Prado \(2018\)](#), this happens when within the 5 minutes of observations volatility towards the end is higher than in other time bins, which may indicate growing volatility—a form of market regime indicator. Again, this is seemingly what the model was able to pick up on and utilize. Intuitively, growing volatility may indicate a greater probability of an extreme drop in price.

While it has been noted by the researchers who introduced these measures that the underlying assumptions can fail in real data, and they explicitly noted how to deal with such cases, it is a genuine novelty that such "abnormal" bid-ask spread estimate values carry additional information that is useful for identifying risky assets. As both of the signed spread estimates have clear interpretation as market regime indicators, my study therefore showcases a novel application of these well-established measures to non-standard financial data.

Another interesting observation that can be made is the discrepancy between the expected effect of adding VPIN and the observed effect, demonstrated on the SHAP scatter plot. As VPIN was designed to serve as an estimate of order flow toxicity ([Easley et al., 2012](#)), one would expect that a model predicting a rug pull-like event would assign a higher probability of the proxy event to higher VPIN values. After all, an extreme drop in price within just 1 hour of token's life should signify market manipulation, or at least vulnerability to it, if VPIN works as intended. However, we see the exact opposite pattern: high VPIN values correspond to lower predicted probability of the proxy event, and low VPIN values make the model assign much higher probability. The difference may have been caused by my modifications to the feature: as shown in [Easley et al. \(2016\)](#), bulk classification, as one of the ways to classify trades as buy or sell, is itself important for "discerning trading intentions from market data". As I replaced bulk classification with the classification approach native to AMM markets, VPIN may have lost most of its intended informational content. However, VPIN has also been criticized: [Andersen and Bondarenko \(2014\)](#) demonstrated that it "is a poor predictor of short run volatility", and thus they did not observe the stated properties of VPIN from [Easley et al. \(2012\)](#) on real data. If so, my study may be additional evidence of the metric's inconsistent behavior.

Additional robustness checks are presented in Appendix D. Those include alternative approaches to measuring statistical significance of performance metric changes, alternative features construction and calibration checks. Any extra information, such as SHAP values in other targets and default models, can be found in my GitHub repository ([Kliuchnikov, 2026](#)).

6 Conclusion

This thesis evaluated whether well-established market microstructure metrics, known to have high explanatory value in traditional financial markets, can improve the early detection of rug pull-type adverse events on Solana’s Raydium DEX. Using 32,426 tokens launched in October and early November 2024, I constructed TAQ-like series from on-chain vault deltas and predicted several rug pull proxy targets within the first hour using features computed from the first 5 minutes of trading. I compared the benchmark feature set based on prior literature with an expanded set that adds a normalized Kyle’s lambda, Harris’s variation of Roll’s spread estimate, Amihud’s illiquidity, a variation of VPIN, order imbalance, median effective spread, and Corwin and Schultz’s spread estimate.

In both CatBoost and RandomForest models, the market microstructure extension resulted in statistically significant performance improvements, especially after hyperparameter tuning. While there is variation in the impact of dataset expansion on the two models, one can see improvement across all targets in at least one of the models.

Interpretability results indicate that, while some microstructure features behave consistently with traditional theory, others exhibit novel emerging properties, partly due to the necessary mathematical adaptations and unique properties of early token lifecycle on a DEX. Specifically, allowing bid-ask spread estimates to be negative allows the model to detect the market regime for that token—a novel application for these measures. Out of all of the added metrics, the modified VPIN demonstrates a flipped relationship relative to the standard interpretation, suggesting that either early-token dynamics differ from mature markets, that VPIN requires a more appropriate adaptation in this setting, or VPIN, as noted previously in the literature, is a poor estimate of order flow toxicity.

The framework used in the study is, to the best of my knowledge, the first of its kind, as there has not been a large-scale rug pull prediction study on the Solana blockchain, despite the immense practical utility. The trained models have shown very high performance on various tasks, with ROC AUC reaching above 0.97 in one of the settings. Having such high model efficiency, market microstructure measures were still able to produce additional statistically significant improvement in performance. The results showcase how theoretically justified features are capable of improving models in financial ML.

I also demonstrated how a practitioner, such as an institutional trader, may extract TAQ-like series from raw on-chain data in order to construct a risk assessment model, as well as important methodological steps that are necessary for reliable validation outcomes, which have been missed in other studies. As an addition, appendix D.2 describes an alternative approach to feature construction, exploring how, when and why it may produce even greater performance on different data.

Overall, the results of the study offer actionable utility for the decentralized finance industry in general. For instance, DEX aggregators, such as DEXScreener, can provide

market microstructure measures, or trained rug pull probability scores, for its users, in order to facilitate risk management for retail investors on the Solana blockchain. Moreover, tokens with high assessed risk may be labeled by the screening system, signaling significant risk to investors, or even black-listing the token altogether. Additionally, this study provides a reproducible pipeline for the development of risk assessment models that institutional DEX investors may use for better adjustment of their risk exposure and avoiding high-risk investments.

The key limitation remains to be the reliance on proxy targets. While some papers tried applying audit-based labeling of tokens, there is still need for an objective ground truth label of a rug pull. Future work should expand on this work by studying a larger sample of data, implement a robust Pump.fun graduation indicator, expand the dataset further or change the architecture, maintaining simplicity in replication and usage, as well as explore the effect of dataset expansion on business metrics (such as expected returns achieved with a decision-making model that uses the proxy models' scores).

As it is obligatory to note, this thesis was made with the usage of LLMs (OpenAI ChatGPT and Google Gemini) for search of academic literature, improvement of the academic style, grammar and spelling corrections, as well as partial translation of my text and annotation.

References

- Akiba, T., S. Sano, T. Yanase, T. Ohta, and M. Koyama (2019). Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, KDD '19, New York, NY, USA, pp. 2623–2631. Association for Computing Machinery.
- Alexander, C., X. Chen, J. Deng, and Q. Fu (2025). Price discovery and efficiency in Uniswap liquidity pools. *Journal of Futures Markets* 45(8), 1023–1048.
- Amihud, Y. (2002). Illiquidity and stock returns: cross-section and time-series effects. *Journal of Financial Markets* 5(1), 31–56.
- Andersen, T. G. and O. Bondarenko (2014). VPIN and the flash crash. *Journal of Financial Markets* 17, 1–46.
- Ashtiani, M. N. and B. Raahemi (2022). Intelligent fraud detection in financial statements using machine learning and data mining: A systematic literature review. *IEEE Access* 10, 72504–72525.
- Aspris, A., A. H. Dyhrberg, S. Foley, W. Krekel, and T. J. Putnins (2026). Is decentralized always better? how market structure affects trading costs for tokenized assets. *Journal of International Financial Markets, Institutions and Money* 109, 102302.
- Breiman, L. (2001). Random forests. *Machine Learning* 45(1), 5–32.
- Capponi, A., R. Jia, and K. Y. Wang (2025). Maximal extractable value and allocative inefficiencies in public blockchains. *Journal of Financial Economics* 172, 104132.
- Chordia, T., R. Roll, and A. Subrahmanyam (2008). Liquidity and market efficiency. *Journal of Financial Economics* 87(2), 249–268.
- Cont, R., A. Kukanov, and S. Stoikov (2014). The price impact of order book events. *Journal of Financial Econometrics* 12(1), 47–88.
- Corwin, S. A. and P. Schultz (2012). A simple way to estimate bid-ask spreads from daily high and low prices. *The Journal of Finance* 67(2), 719–759.
- Deng, S., C. Wang, Z. Fu, and M. Wang (2021). An intelligent system for insider trading identification in Chinese security market. *Computational Economics* 57(2), 593–616.
- Deng, S., C. Wang, J. Li, H. Yu, H. Tian, Y. Zhang, Y. Cui, F. Ma, and T. Yang (2019). Identification of insider trading using extreme gradient boosting and multi-objective optimization. *Information* 10(12), 367.

- Easley, D., N. M. Kiefer, M. O'Hara, and J. B. Paperman (1996). Liquidity, information, and infrequently traded stocks. *The Journal of Finance* 51(4), 1405–1436.
- Easley, D., M. M. López de Prado, and M. O'Hara (2012). Flow toxicity and liquidity in a high-frequency world. *The Review of Financial Studies* 25(5), 1457–1493.
- Easley, D., M. M. López de Prado, and M. O'Hara (2016). Discerning information from trade data. *Journal of Financial Economics* 120(2), 269–285.
- Efron, B. and R. J. Tibshirani (1994). Bootstrap methods and permutation tests. In *An Introduction to the Bootstrap*, Chapter 16, pp. 16–1–16–57. CRC press.
- Golmohammadi, K., O. R. Zaiane, and D. Díaz (2014). Detecting stock market manipulation using supervised learning algorithms. In *2014 International Conference on Data Science and Advanced Analytics (DSAA)*, pp. 435–441.
- Good, P. (2000). *Permutation tests* (2 ed.). Springer Series in Statistics. New York, NY: Springer.
- Goyenko, R. Y., C. W. Holden, and C. A. Trzcinka (2009). Do liquidity measures measure liquidity? *Journal of Financial Economics* 92(2), 153–181.
- Han, J., S. Huang, and Z. Zhong (2024). Trust in DeFi: an empirical study of the decentralized exchange. Available at SSRN: <https://dx.doi.org/10.2139/ssrn.3896461>.
- Harris, L. (1990). Statistical properties of the Roll serial covariance bid/ask spread estimator. *The Journal of Finance* 45(2), 579–590.
- Hasbrouck, J. (2009). Trading costs and returns for U.S. equities: Estimating effective costs from daily data. *The Journal of Finance* 64(3), 1445–1477.
- Hilal, W., S. A. Gadsden, and J. Yawney (2022). Financial fraud: A review of anomaly detection techniques and recent advances. *Expert Systems with Applications* 193, 116429.
- Ilemi (2026). Solana DEX metrics. Dune dashboard. <https://dune.com/ilemi/solana-dex-metrics>. Accessed: 2026-03-01.
- Jia, R. (2024). *The Market Microstructure of Decentralized Exchanges*. Ph.D. thesis, Columbia University.
- Kalacheva, A., P. Kuznetsov, I. Vodolazov, and Y. Yanovich (2025). Detecting rug pulls in decentralized exchanges: The rise of meme coins. *Blockchain: Research and Applications* 6, 100336.

- Khalili, J. (2024). A kid made \$50,000 dumping crypto he'd created. Then came the backlash. <https://www.wired.com/story/memecoin-kid-backlash/>. Published December 6, 2024.
- Khalili, J. (2025). The madcap rise of memecoin factory Pump.Fun. <https://www.wired.com/story/madcap-rise-of-memecoin-factory-pumpfun/>. Published January 15, 2025.
- Kliuchnikov, N. (2026). Rug-pull-early-detection-bachelor-thesis-. GitHub repository (source code). <https://github.com/NickLithan/Rug-Pull-Early-Detection--Bachelor-Thesis->.
- Koehn, P. (2004). Statistical significance tests for machine translation evaluation. In D. Lin and D. Wu (Eds.), *Proceedings of the 2004 Conference on Empirical Methods in Natural Language Processing*, Barcelona, Spain, pp. 388–395. Association for Computational Linguistics.
- Kreiss, J.-P. and S. N. Lahiri (2012). 1 – Bootstrap methods for time series. In T. Subba Rao, S. Subba Rao, and C. Rao (Eds.), *Time Series Analysis: Methods and Applications*, Volume 30 of *Handbook of Statistics*, pp. 3–26. Elsevier.
- Kyle, A. S. (1985). Continuous auctions and insider trading. *Econometrica* 53(6), 1315–1335.
- Kyle, A. S. and A. A. Obizhaeva (2016). Market microstructure invariance: Empirical hypotheses. *Econometrica* 84(4), 1345–1404.
- La Morgia, M., A. Mei, F. Sassi, and J. Stefa (2023). The Doge of Wall Street: Analysis and detection of pump and dump cryptocurrency manipulations. *ACM Trans. Internet Technol.* 23(1), 11:1–11:28.
- Lehar, A. and C. Parlour (2025). Decentralized exchange: The Uniswap automated market maker. *The Journal of Finance* 80(1), 321–374.
- Lin, Z., J. Chen, J. Wu, W. Zhang, Y. Wang, and Z. Zheng (2024). CRPWarner: Warning the risk of contract-related rug pull in DeFi smart contracts. *IEEE Trans. Softw. Eng.* 50(6), 1534–1547.
- Lipton, Z. C., C. Elkan, and B. Naryanaswamy (2014). Optimal thresholding of classifiers to maximize F1 measure. In T. Calders, F. Esposito, E. Hüllermeier, and R. Meo (Eds.), *Machine Learning and Knowledge Discovery in Databases*, Berlin, Heidelberg, pp. 225–239. Springer Berlin Heidelberg.

- Liu, Q., C. Wang, P. Zhang, and K. Zheng (2021). Detecting stock market manipulation via machine learning: Evidence from China Securities Regulatory Commission punishment cases. *International Review of Financial Analysis* 78, 101887.
- Lopez de Prado, M. (2018). *Advances in financial machine learning*. Nashville, TN: John Wiley & Sons.
- Lundberg, S. M. and S.-I. Lee (2017). A unified approach to interpreting model predictions. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*, NIPS’17, Red Hook, NY, USA, pp. 4768–4777. Curran Associates Inc.
- Ma, F., M. Ren, L. Ouyang, Y. Chen, J. Zhu, T. Chen, Y. Zheng, X. Dai, Y. Jiang, and J. Sun (2023). Pied-Piper: Revealing the backdoor threats in Ethereum ERC token contracts. *ACM Trans. Softw. Eng. Methodol.* 32(3), 1–24.
- Mazorra, B., V. Adan, and V. Daza (2022). Do not rug on me: Leveraging machine learning techniques for automated scam detection. *Mathematics* 10(6), 949.
- Mazzarisi, P., A. Ravagnani, P. Deriu, F. Lillo, F. Medda, and A. Russo (2022). A machine learning approach to support decision in insider trading detection. <https://doi.org/10.48550/arXiv.2212.05912>. arXiv preprint.
- Merkle Science (2025). Weaponized influence: Social engineering in crypto scams. <https://www.merklescience.com/blog/weaponized-influence-social-engineering-in-crypto-scams>. Blog post, published February 11, 2025.
- O’Hara, M. (1997). *Market Microstructure Theory*. Blackwell Publishers.
- Prokhorenkova, L., G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin (2018). Cat-Boost: unbiased boosting with categorical features. In *Proceedings of the 32nd International Conference on Neural Information Processing Systems*, NIPS’18, Red Hook, NY, USA, pp. 6639–6649. Curran Associates Inc.
- Pump Fun (2026). pump-public-docs. GitHub repository (source code). <https://github.com/pump-fun/pump-public-docs>.
- Raydium Protocol (2025). Say Hello to Raydium. <https://docs.raydium.io/>. Raydium documentation, last updated 2025.
- Roll, R. (1984). A simple implicit measure of the effective bid-ask spread in an efficient market. *The Journal of Finance* 39(4), 1127–1139.

- Saito, T. and M. Rehmsmeier (2015). The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLOS ONE* 10(3), 1–21.
- Solana Foundation (2019). How Solana’s proof of history is a huge advancement for block time. <https://solana.com/news/how-solana-s-proof-of-history-is-a-huge-a-dvancement-for-block-time>. Published August 13, 2019.
- Solana Foundation (2026a). A complete guide to Solana development for Ethereum developers. <https://solana.com/developers/evm-to-svm/complete-guide>. Section “Where is the Mempool?”. Accessed January 2026.
- Solana Foundation (2026b). Understanding Solana transaction fees. <https://solana.com/learn/understanding-solana-transaction-fees>. Accessed January 2026.
- Solidus Labs Research (2025). The 2025 rug pull report: Rug pulls and pump-and-dumps on Solana. Technical report, Solidus Labs. <https://www.soliduslabs.com/reports/solana-rug-pulls-pump-dumps-crypto-compliance>.
- Srifa, S., Y. Yanovich, R. Vasilyev, T. Rupasinghe, and V. Amelin (2025). Rug pull detection on decentralized exchange using transaction data. *Blockchain: Research and Applications* 6(3), 100275.
- Sun, D., W. Ma, L. Nie, and Y. Liu (2024). SoK: Comprehensive analysis of rug pull causes, datasets, and detection tools in DeFi. <https://arxiv.org/abs/2403.16082>. arXiv preprint.
- Sun, X. and W. Xu (2014). Fast implementation of DeLong’s algorithm for comparing the areas under correlated receiver operating characteristic curves. *IEEE Signal Processing Letters* 21(11), 1389–1393.
- Wang, Q., W. Xu, X. Huang, and K. Yang (2019). Enhancing intraday stock price manipulation detection by leveraging recurrent neural networks with ensemble learning. *Neurocomputing* 347, 46–58.
- Wu, C., J. Chen, Z. Zhao, K. He, G. Xu, Y. Wu, H. Wang, H. Li, Y. Liu, and Y. Xiang (2024). TokenScout: Early detection of Ethereum scam tokens via temporal graph learning. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security, CCS ’24*, New York, NY, USA, pp. 956–970. Association for Computing Machinery.
- Wu, H., H. Wang, S. Li, Y. Wu, M. Fan, W. Jin, and T. Liu (2025). RPHunter: Unveiling rug pull schemes in crypto token via code-and-transaction fusion analysis. <https://doi.org/10.48550/arXiv.2506.18398>. arXiv preprint.

- Xia, P., H. Wang, B. Gao, W. Su, Z. Yu, X. Luo, C. Zhang, X. Xiao, and G. Xu (2021). Trade or Trick? Detecting and characterizing scam tokens on Uniswap decentralized exchange. *Proc. ACM Meas. Anal. Comput. Syst.* 5(3), 1–26.
- Yaremus, D., J. Li, A. Kalacheva, I. Vodolazov, and Y. Yanovich (2025). Detecting rug pulls in decentralized exchanges: machine learning evidence from the TON blockchain. <https://doi.org/10.48550/arXiv.2509.01168>. arXiv preprint.
- Yeh, A. (2000). More accurate tests for the statistical significance of result differences. In *Proceedings of the 18th Conference on Computational Linguistics - Volume 2*, COLING '00, USA, pp. 947–953. Association for Computational Linguistics.
- Zhu, B. Z., D. Liu, X. Wan, G. Liao, C. C. Moallemi, and B. Bachu (2024). What drives liquidity on decentralized exchanges? Evidence from the Uniswap protocol. <https://doi.org/10.48550/arXiv.2410.19107>. arXiv preprint.

Appendices

Appendix A Benchmark Features

Feature	Description
n_transfers	Total number of transactions with transfers from/to liquidity pool vaults
n_swaps	Number of swaps (opposite sign of vault deltas)
n_buys	Number of buys (swap, base vault decreased)
n_lp_addition_like	Number of transactions that increase both vault account sizes
n_lp_removal_like	Number of transactions that decrease both vault account sizes
buy_percentage	Percent of transactions that were token buys
sell_percentage	Percent of transactions that were token sells
buy_sell_ratio	Ratio of the number of buys to the number of sells
init_price	Initial midquote of the token
max_price	Peak midquote value
min_price	Lowest midquote value
std_price	Standard deviation in midquote
std_buy_price	Standard deviation in effective price of buys
std_sell_price	Standard deviation in effective price of sells
return_1min	Percent change of midquote within 1 minute from launch
return_5min	Percent change of midquote within observation window
supply	Initial amount of tokens in the base vault
init_tvl_sol	Initial amount of WSOL in the quote vault
max_volume	Maximum number of tokens traded within one trade
min_volume	Minimum number of tokens traded within one trade
max_wsol_volume	Maximum amount of WSOL traded within one trade
min_wsol_volume	Minimum amount of WSOL traded within one trade
total_wsol_turnover	Total amount of WSOL traded
total_wsol_buy_turnover	Total amount of WSOL paid in buys
wsol_turnover_1min	Amount of WSOL traded within 1 minute from launch
wsol_buy_turnover_1min	Amount of WSOL paid in buys within 1 minute

Table 4: Benchmark features: part 1

Part 2/2:

Feature	Description
time_deployemnt2trade	Seconds from pool deployment to first trade
time2max_volume	Seconds from launch to max_volume occurrence
time2max_wsol_volume	Seconds from launch to max_wsol_volume occurrence
first_buy_rel_ts	Seconds from launch to first buy
first_sell_rel_ts	Seconds from launch to first sell
avg_rsi_{N}	Average of Relative Strength Index on 10-second time bars with smoothing parameter N (N in {6, 12})
std_rsi_{N}	Standard deviation in Relative Strength Index

Table 5: Benchmark features: part 2

Appendix B SHAP with has_pumpdotfun_history

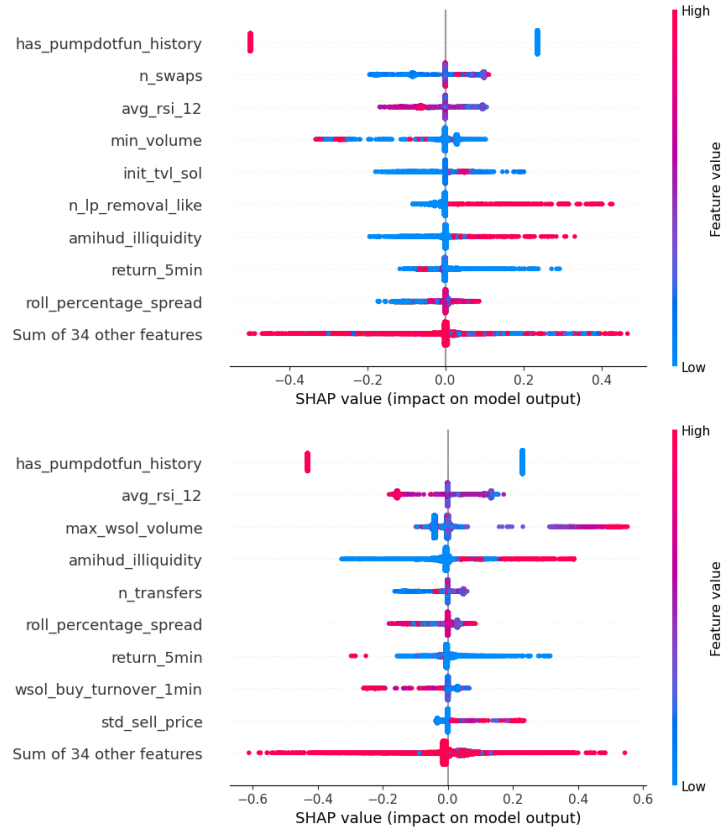


Figure 4: SHAP values for tuned RandomForestClassifier with has_pumpdotfun_history. Top model predicts md_liq_no_post_rc_0.99_0.01, the bottom – md_liq_0.9.

Appendix C Hyperparameters

Here I present all hyperparameters optimized in the hyperparameter tuning. The option space was selected in such a way that we can reasonably assume that the model was trained in the vicinity of the theoretically optimal hyperparameter combination.

Model	Hyperparameter	Option Space
CatBoostClassifier	depth	[5, 10]
	learning_rate	[3e-3, 0.05], log scale
	l2_leaf_reg	[0.1, 10.0], log scale
	colsample_bylevel	[0.3, 1.0]
	min_child_samples	[5, 100]
	random_strength	[1e-2, 10.0], log scale
RandomForestClassifier	n_estimators	[300, 1000]
	max_depth	[10, 18]
	min_samples_split	[2, 100]
	min_samples_leaf	[1, 50]
	max_features	[0.5, 1.0]

Table 6: Hyperparameters used in Optuna hyperparameter tuning.

The number of Optuna optimization steps was set to 40 for both models. The maximized value was TimeSeriesSplit (5 splits) validation ROC AUC. For CatBoost specifically the number of estimators was capped at 1500 and the best iteration was picked based on validation ROC AUC. The final CatBoost model was trained with the optimal number of estimators for the last CV split (first train on first 4 folds, use 5th fold as validation to pick the best estimator number) on the entire training data.

Appendix D Robustness

D.1 Statistical tests of performance metric change

While DeLong’s algorithm (specific to ROC AUC) and paired bootstrap are some of the standard ways to estimate, under the null hypothesis, how likely a difference in model performance at least as large as the observed one would be, there exist alternative approaches that may be as valid or possibly more correct in this problem. It may be that under a different choice of statistical tests the p-values would be different.

So, let us discuss the details of statistical tests on change in performance metric. For several such tests the formulation goes as such (Efron and Tibshirani, 1994). Let the population of features and targets be over the sets $\mathcal{X}$ and $\mathcal{Y}$, respectively. Let the (random) test dataset of size n be (X, Y) with $X \in \mathcal{X}^n$ and $Y \in \mathcal{Y}^n$. Let m_1 and m_2 be the two models we are comparing with $m_i : \mathcal{X} \rightarrow \hat{\mathcal{Y}}$ and $m_i(X) := (m_i(X_1), \dots, m_i(X_n))$.

Finally, let $f : \mathcal{Y}^n \times \hat{\mathcal{Y}}^n \rightarrow \mathbb{R}$ be the performance metric. We would like to test

$$H_0 : \mathbb{E}[T] = 0, \quad H_1 : \mathbb{E}[T] \neq 0, \quad T := f(Y, m_1(X)) - f(Y, m_2(X)) \quad (23)$$

The idea is to generate a "null" distribution F —estimate of the sample distribution that a pair of models with indistinguishable performance would produce. So, if T_{obs} is the observed difference, and we sample values $\tilde{T}_j \sim F$, then the p-value in such a test is

$$p := \hat{\mathbb{P}}_{\tilde{T}_j \sim F}(|\tilde{T}_j| \geq |T_{obs}|), \quad (24)$$

after which we use the p-value like normal to reject/fail to reject the null hypothesis.

One way to generate the null distribution is to use paired bootstrap (Koehn, 2004). First, generate bootstrap sample (X', Y') many times, as in Efron and Tibshirani (1994). The null distribution will be the sample distribution of

$$\tilde{T}_j := T' - T_{obs} \sim F, \quad T' := f(Y', m_1(X')) - f(Y', m_2(X')) \quad (25)$$

The intuition behind this test is that the variation produced by bootstrap resampling should be similar to that of sampling-caused variation between truly indistinguishable models, so if the difference we observe is unlikely under the null, it should also be an outlier among the paired bootstrap-produced offsets.

Importantly, an alternative approach is also possible. Instead, paired permutation test is often used in machine learning (Good, 2000; Yeh, 2000). The idea is that we can assume that under the null hypothesis, the predictions of m_1 and m_2 are interchangeable (not necessarily true). Thus, we can shuffle the predictions of each model to generate the null distribution. Specifically, construct arrays $\hat{Y}'_1$ and $\hat{Y}'_2$, where

$$\forall i \in \{1, \dots, n\} : ((\hat{Y}'_1)_i, (\hat{Y}'_2)_i) := \sigma_i(m_1(X_i), m_2(X_i)), \quad (26)$$

where $\sigma_i : \hat{\mathcal{Y}} \times \hat{\mathcal{Y}} \rightarrow \hat{\mathcal{Y}} \times \hat{\mathcal{Y}}$ randomly swap the two models' predictions within each observation (independently with probability 1/2). This way the null distribution is generated by

$$\tilde{T}_j := f(Y, \hat{Y}'_1) - f(Y, \hat{Y}'_2) \sim F \quad (27)$$

Finally, if observations exhibit temporal dependence or clustering, the assumptions of the bootstrap approach fail, specifically that observations (X_i, Y_i) are i.i.d. This is quite likely, considering the nature of the financial data studied. In this case one should use an alternative approach, such as block bootstrap, for generating the null distribution (Kreiss and Lahiri, 2012). However, if the time-series dependence is insignificant, the

block bootstrap tends to be too conservative.

In Tables 7 and 8 I present the p-values under several approaches for the model comparison in section 5. The Default stands for p-values of DeLong’s algorithm for ROC AUC and paired bootstrap for PR AUC and F1 score, Permutation is the paired permutation test, and BlockBStrap is paired block bootstrap (block size is 100). I underlined the p-values whose corresponding confidence level differs from the default one for each specification. As we can see, the p-values are very similar, and the differences in confidence levels do not alter the fact that the new features allow for truly statistically significant improvement in models’ predictions.

D.2 Cross-sectional Feature Construction

Market microstructure metrics are often used as detection tools of anomalies in liquidity and architecture. Therefore, it is natural to replace the metric levels with standardized values, such as percentiles and z-statistics. However, as I am using tree-based ML models, any monotonous transformations are irrelevant, as the models only depend on the rankings between feature values.

Nonetheless, one way my approach could be enhanced is by applying a cross-sectional approach. Essentially, there is a possibility that with the development of the market and change in market regimes the distribution of the feature values shifts, thus the same feature value may mean different things as time passes by. One way to track this change, and at the same time replace the market microstructure metrics’ levels with interpretable standardized values, is to replace them with the rolling quantiles.

The transformation is quite simple: track a 2-day rolling window of observed token launches (1000-2000 observations in each window) and calculate the quantile of the feature value among observed values within that window. This way we essentially apply a cross-sectional transformation (differs from the standard cross-sectional approach used for low frequency data) that standardizes the data to values between 0 and 1 without look-ahead bias, which is crucial for financial data. I also dropped the first 2 days of the training data, as the rolling window could not be built for those observations.

Tables 9 and 10 report the results when we replace new features with their rolling quantiles (Q-Expanded), compared to the initial approach. As one can see, in most settings the models are equally effective, and in several cases the performance actually dropped with the transformation. The reason why this is likely happening is because when we use rolling quantiles, there is a trade-off between detecting inter-temporal dynamic in rug pull-like event patterns and introducing noise caused by a relatively smaller number of observations within the rolling window. As evidence of this mechanism, the tuned CatBoostClassifier model, which should effectively exploit the uncovered intertemporal dynamic while not necessarily combating the noise in the features, only had decreased

Target	P-value Type	ROC AUC	PR AUC	F1 Score
CatBoostClassifier (default)				
$MD_p \geq 90\%$ & $RC_p \leq 1\%$	Default	0.0161 *	0.0531 ·	0.2807
	Permutation	0.0180 *	0.0569 ·	0.2802
	BlockBStrap	0.0276 *	0.0629 ·	0.2725
$MD_p \geq 99\%$ & $RC_p \leq 0.5\%$	Default	0.3663	0.4683	0.4082
	Permutation	0.3619	0.4690	0.3996
	BlockBStrap	0.3479	0.4670	0.3793
$MD_L \geq 99\%$ & $RC_L \leq 1\%$	Default	0.1973	0.0078 **	0.4184
	Permutation	0.1945	0.0059 **	0.4131
	BlockBStrap	0.2162	0.0087 **	0.4254
$MD_L \geq 90\%$	Default	0.1438	0.0536 ·	0.3695
	Permutation	0.1424	0.0534 ·	0.3890
	BlockBStrap	0.1646	0.0563 ·	0.3418
$\delta T \geq 5min.$	Default	0.1115	0.1278	0.0812 ·
	Permutation	0.1152	0.1367	0.0742 ·
	BlockBStrap	0.1562	0.1612	<u>0.1202</u>
RandomForestClassifier (default)				
$MD_p \geq 90\%$ & $RC_p \leq 1\%$	Default	0.0022 **	0.0006 ***	0.2509
	Permutation	0.0023 **	0.0005 ***	0.2442
	BlockBStrap	0.0012 **	0.0004 ***	0.2365
$MD_p \geq 99\%$ & $RC_p \leq 0.5\%$	Default	0.0058 **	0.0003 ***	0.0164 *
	Permutation	0.0070 **	0.0004 ***	0.0168 *
	BlockBStrap	<u>0.0131</u> *	<u>0.0018</u> **	0.0180 *
$MD_L \geq 99\%$ & $RC_L \leq 1\%$	Default	0.2115	0.1983	0.1166
	Permutation	0.2082	0.2002	0.1136
	BlockBStrap	0.2531	0.2150	0.1419
$MD_L \geq 90\%$	Default	0.0048 **	0.0155 *	0.8714
	Permutation	0.0034 **	0.0161 *	0.8730
	BlockBStrap	0.0081 **	0.0176 *	0.8822
$\delta T \geq 5min.$	Default	0.0014 **	0.0014 **	0.6007
	Permutation	<u>0.0006</u> ***	<u>0.0004</u> ***	0.6048
	BlockBStrap	<u>0.0006</u> ***	<u>0.0006</u> ***	0.5846

Table 7: P-values for change in performance metrics of default CatBoostClassifier (top) and RandomForestClassifier (bottom). Significance codes:
· $p \in [0.05, 0.1]$; * $p \in [0.01, 0.05]$; ** $p \in [0.001, 0.01]$; *** $p < 0.001$.

Target	P-value Type	ROC AUC	PR AUC	F1 Score
CatBoostClassifier (tuned)				
$MD_p \geq 90\%$ & $RC_p \leq 1\%$	Default	0.0125 *	0.0093 **	0.3912
	Permutation	0.0135 *	<u>0.0106</u> *	0.3851
	BlockBStrap	<u>0.0095</u> **	0.0094 **	0.4260
$MD_p \geq 99\%$ & $RC_p \leq 0.5\%$	Default	0.1907	0.6602	0.4350
	Permutation	0.1911	0.6598	0.4389
	BlockBStrap	0.1639	0.6704	0.3982
$MD_L \geq 99\%$ & $RC_L \leq 1\%$	Default	0.0048 **	0.0092 **	0.3578
	Permutation	0.0091 **	<u>0.0119</u> *	0.3487
	BlockBStrap	0.0050 **	<u>0.0110</u> *	0.3250
$MD_L \geq 90\%$	Default	0.0034 **	0.0002 ***	0.2858
	Permutation	0.0038 **	0.0006 ***	0.2905
	BlockBStrap	0.0042 **	0.0003 ***	0.2646
$\delta T \geq 5min.$	Default	0.0008 ***	0.0126 *	0.6166
	Permutation	0.0007 ***	0.0153 *	0.6267
	BlockBStrap	<u>0.0061</u> **	0.0327 *	0.6278
RandomForestClassifier (tuned)				
$MD_p \geq 90\%$ & $RC_p \leq 1\%$	Default	0.0099 **	0.0034 **	0.5849
	Permutation	<u>0.0132</u> *	0.0040 **	0.5756
	BlockBStrap	<u>0.0148</u> *	0.0065 **	0.5749
$MD_p \geq 99\%$ & $RC_p \leq 0.5\%$	Default	1.1e-05 ***	0.0002 ***	0.4260
	Permutation	0.0000 ***	0.0003 ***	0.4282
	BlockBStrap	0.0000 ***	0.0005 ***	0.4153
$MD_L \geq 99\%$ & $RC_L \leq 1\%$	Default	0.4814	0.2201	0.4194
	Permutation	0.4828	0.3142	0.4277
	BlockBStrap	0.5321	0.2434	0.4712
$MD_L \geq 90\%$	Default	0.0004 ***	0.0000 ***	0.4877
	Permutation	0.0009 ***	0.0000 ***	0.4938
	BlockBStrap	0.0009 ***	0.0002 ***	0.4754
$\delta T \geq 5min.$	Default	0.8785	0.1791	0.0903 .
	Permutation	0.8797	0.1670	0.0866 .
	BlockBStrap	0.8816	0.2021	<u>0.1098</u>

Table 8: P-values for change in performance metrics of CatBoostClassifier (top) and RandomForestClassifier (bottom) with hyperparameter tuning. Significance codes: . $p \in [0.05, 0.1]$; * $p \in [0.01, 0.05]$; ** $p \in [0.001, 0.01]$; *** $p < 0.001$.

performance due to the transformation. Meanwhile, the tuned RandomForestClassifier model, which is designed to decrease noise in predictions through bagging, did not have such a drop, but also did not have significant improvement, due to seemingly small relevance of intertemporal dynamics.

Overall, if I studied a significantly longer period of observations, it is very likely that such a transformation would be necessary to detect shifts in the feature distributions due to market dynamics. Increasing the window size reduces the train data size (as early on there is not enough data to fill the rolling window), but on longer that is not only feasible, but also beneficial for reducing the noisiness of the construction. Therefore, this transformation is likely to be useful in future research.

D.3 Model Calibration and Correctness of F1 score

As Figure 5 shows, the (tuned) models are not entirely calibrated—their predicted "probabilities" of rug pull-like events do not represent the actual implied probability. While rank metrics, such as ROC AUC and PR AUC do not require calibration, F1 score depends on it directly—it selects predicted class via a pre-specified threshold on the predicted probability, normally set to 0.5 (which is what I used in this study). As [Lipton et al. \(2014\)](#) show, miscalibration of the model can result in F1 score with fixed threshold working incorrectly. That is why a truly improved model can have no significant increase in F1 score, or even a lower score. The reason why I still used the standard F1 score is that this was the main performance metric for prior research in rug pull identification, and intentionally removing it due to lack of observed change could artificially inflate observed level of statistical significance.

Target	Model	ROC AUC	PR AUC	F1 Score
CatBoostClassifier (default)				
$MD_p \geq 90\%$ & $RC_p \leq 1\%$	Expanded	0.8721	0.8785	0.7851
	Q-Expanded	0.8725	0.8777	0.7833
	(<i>p-value</i>)	(0.5755)	(0.2977)	(0.3956)
$MD_p \geq 99\%$ & $RC_p \leq 0.5\%$	Expanded	0.9436	0.8723	0.7953
	Q-Expanded	0.9438	0.8739	0.7926
	(<i>p-value</i>)	(0.6185)	(0.2281)	(0.3357)
$MD_L \geq 99\%$ & $RC_L \leq 1\%$	Expanded	0.9696	0.9488	0.8671
	Q-Expanded	0.9706	0.9500	0.8712
	(<i>p-value</i>)	(0.0159) *	(0.0757) ·	(0.0858) ·
$MD_L \geq 90\%$	Expanded	0.9619	0.9481	0.8619
	Q-Expanded	0.9618	0.9482	0.8627
	(<i>p-value</i>)	(0.8366)	(0.8139)	(0.6570)
$\delta T \geq 5min.$	Expanded	0.8998	0.9047	0.8129
	Q-Expanded	0.8996	0.9047	0.8119
	(<i>p-value</i>)	(0.7697)	(0.9457)	(0.6309)
RandomForestClassifier (default)				
$MD_p \geq 90\%$ & $RC_p \leq 1\%$	Expanded	0.8649	0.8697	0.7832
	Q-Expanded	0.8667	0.8709	0.7855
	(<i>p-value</i>)	(0.1039)	(0.3435)	(0.3660)
$MD_p \geq 99\%$ & $RC_p \leq 0.5\%$	Expanded	0.9425	0.8650	0.7899
	Q-Expanded	0.9423	0.8643	0.7806
	(<i>p-value</i>)	(0.8624)	(0.7017)	(0.0080) **
$MD_L \geq 99\%$ & $RC_L \leq 1\%$	Expanded	0.9684	0.9416	0.8649
	Q-Expanded	0.9693	0.9438	0.8674
	(<i>p-value</i>)	(0.0134) *	(0.0579) ·	(0.2595)
$MD_L \geq 90\%$	Expanded	0.9602	0.9426	0.8583
	Q-Expanded	0.9598	0.9429	0.8605
	(<i>p-value</i>)	(0.5354)	(0.7766)	(0.3499)
$\delta T \geq 5min.$	Expanded	0.8951	0.8988	0.8117
	Q-Expanded	0.8952	0.8976	0.8088
	(<i>p-value</i>)	(0.9032)	(0.2757)	(0.2035)

Table 9: Comparison of feature construction approaches with performance metrics of default CatBoostClassifier (top) and RandomForestClassifier (bottom). Significance codes: · $p \in [0.05, 0.1]$; * $p \in [0.01, 0.05]$; ** $p \in [0.001, 0.01]$; *** $p < 0.001$.

Target	Model	ROC AUC	PR AUC	F1 Score
CatBoostClassifier (tuned)				
$MD_p \geq 90\%$ & $RC_p \leq 1\%$	Expanded	0.8725	0.8794	0.7866
	Q-Expanded	0.8706	0.8778	0.7851
	(<i>p-value</i>)	(4.6e-05) ***	(0.0010) ***	(0.4016)
$MD_p \geq 99\%$ & $RC_p \leq 0.5\%$	Expanded	0.9449	0.8742	0.7944
	Q-Expanded	0.9449	0.8742	0.7941
	(<i>p-value</i>)	(0.8479)	(0.9658)	(0.8991)
$MD_L \geq 99\%$ & $RC_L \leq 1\%$	Expanded	0.9716	0.9515	0.8729
	Q-Expanded	0.9706	0.9504	0.8736
	(<i>p-value</i>)	(0.0013) **	(0.0327) *	(0.6361)
$MD_L \geq 90\%$	Expanded	0.9630	0.9496	0.8641
	Q-Expanded	0.9624	0.9492	0.8633
	(<i>p-value</i>)	(0.0547) ·	(0.3414)	(0.6084)
$\delta T \geq 5min.$	Expanded	0.9012	0.9065	0.8130
	Q-Expanded	0.9015	0.9065	0.8135
	(<i>p-value</i>)	(0.5259)	(0.9993)	(0.7856)
RandomForestClassifier (tuned)				
$MD_p \geq 90\%$ & $RC_p \leq 1\%$	Expanded	0.8662	0.8744	0.7781
	Q-Expanded	0.8669	0.8739	0.7814
	(<i>p-value</i>)	(0.2861)	(0.5043)	(0.0870) ·
$MD_p \geq 99\%$ & $RC_p \leq 0.5\%$	Expanded	0.9447	0.8712	0.7875
	Q-Expanded	0.9441	0.8700	0.7920
	(<i>p-value</i>)	(0.1847)	(0.1944)	(0.1041)
$MD_L \geq 99\%$ & $RC_L \leq 1\%$	Expanded	0.9706	0.9478	0.8728
	Q-Expanded	0.9710	0.9480	0.8721
	(<i>p-value</i>)	(0.0986) ·	(0.7850)	(0.6596)
$MD_L \geq 90\%$	Expanded	0.9619	0.9476	0.8638
	Q-Expanded	0.9621	0.9483	0.8643
	(<i>p-value</i>)	(0.5598)	(0.1214)	(0.7804)
$\delta T \geq 5min.$	Expanded	0.8972	0.9031	0.8114
	Q-Expanded	0.8971	0.9021	0.8107
	(<i>p-value</i>)	(0.9546)	(0.0908) ·	(0.6603)

Table 10: Comparison of feature construction approaches with performance metrics of CatBoostClassifier (top) and RandomForestClassifier (bottom) with tuned hyperparameters. Significance codes: · $p \in [0.05, 0.1]$; * $p \in [0.01, 0.05]$; ** $p \in [0.001, 0.01]$; *** $p < 0.001$.

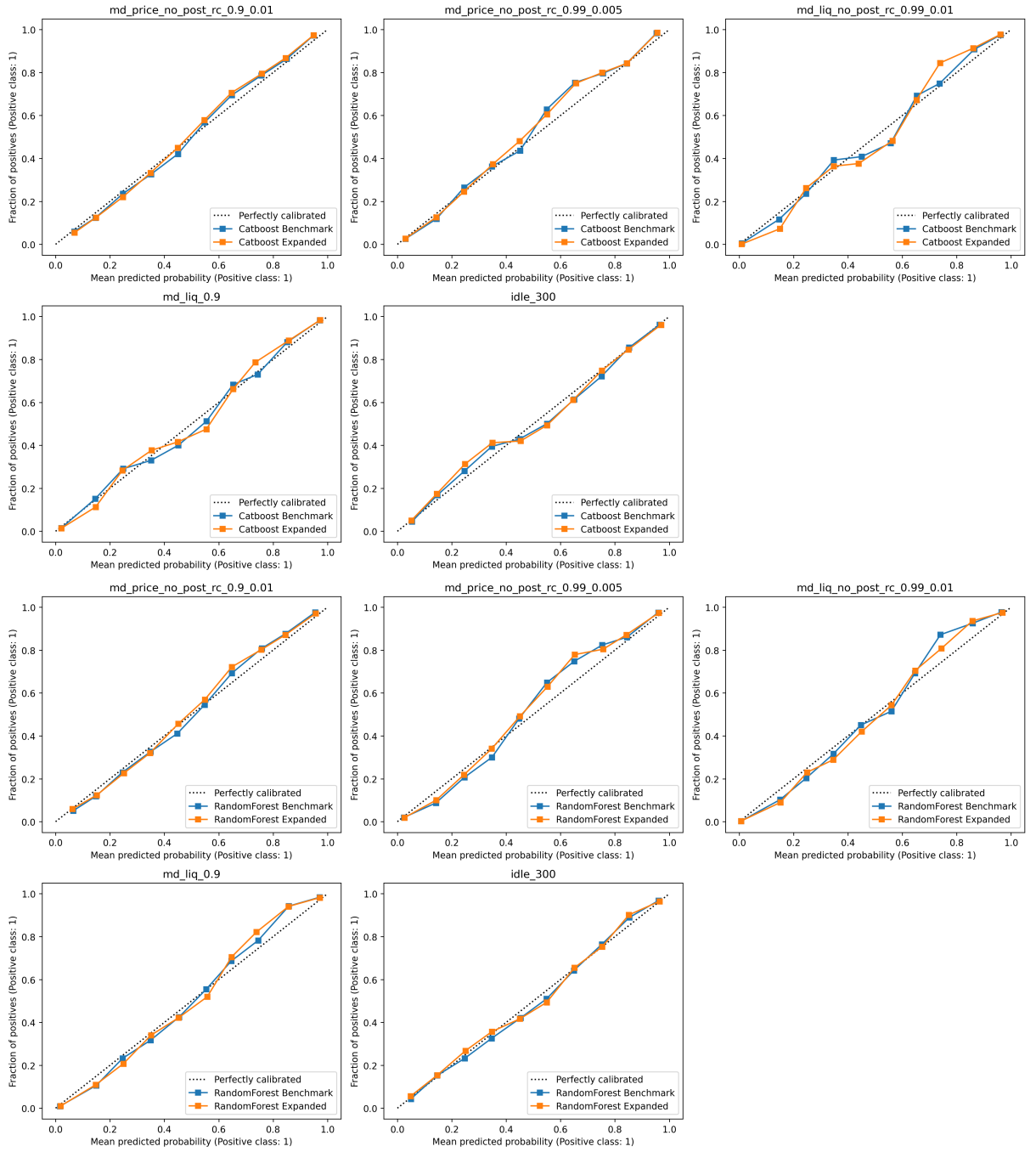


Figure 5: Calibration plots for tuned CatBoostClassifier (top) and RandomForestClassifier (bottom).